

Laboratory Manual: High-Level Synthesis

Advanced Computer Architecture WT25

Univ.Prof. Dr.-Ing. Dipl.-Ing. Daniel Müller-Gritschneider
Kappes Johannes, M.Sc.

November 14, 2025

Embedded Computing Systems
TU Wien Informatics
Treitlstr. 3, 1040 Wien
Tel.: (089) 289 23666

©Further publication only with the consent of the institute. Thank you to Johannes Geier, Prof. Ulf Schlichtmann, and the TUM EDA chair for the distribution. Additionally, the assistance of Florian Huemer and Heinz Deinhart was greatly appreciated.

Contents

1	Lab Module 1: Virtual Prototyping	1
1.1	Virtual Prototyping	1
1.2	EDAduino Virtual Platform	1
1.2.1	System on Chip	1
1.3	Toolchain	4
1.4	Hello World! Tutorial	5
1.4.1	Getting Started	5
1.5	IDE	10
1.6	Task 1: Driving UART on a Bare-metal System	14
1.6.1	Complete UART Send String Function	14
1.6.2	Wait for Interrupt	14
1.7	Questions (for practice only)	15
1.8	Appendix	15
1.8.1	Task 1:	15
1.9	FAQ	17
2	Lab Module 2: High Level Synthesis with Vivado-HLS	19
2.0.1	Preparation	19
2.0.2	Part 1 - Introduction	19
2.0.3	Part 2 - Design Analysis	20
2.1	Questions (for practice only)	20
2.2	Appendix	21
2.2.1	FAQ	21
2.2.2	Reference Values	21
3	Lab Module 3: Advanced Encryption Standard	22
3.0.1	AES Block Cipher	22
3.0.2	Block Cipher Mode of Operation	22
3.0.3	AES Hardware Accelerator	23
3.0.4	Getting Started	26
3.1	Laboratory Tasks	28
3.1.1	A) Application Profiling of Pure Software Implementation (AES-SW)	28
3.1.2	B) AES Hardware Accelerator without Direct Memory Access	31
3.1.3	C) AES Hardware Accelerator with Direct Memory Access	38
3.2	Questions (for exercise only)	45
3.3	Appendix	45
3.3.1	FAQ	45

Bibliography

47

1 Lab Module 1: Virtual Prototyping

1.1 Virtual Prototyping

Traditionally, software (SW) is developed on finished hardware (HW). This is typically the case for most conventional development of digital systems. However, a drawback is that iterations in the development process must be consecutive; that is, a company's software department must wait for the hardware department to launch the new computational platform. While writing software that mostly avoids platform dependencies might be a good practice, accounting for the hardware platform beneath cannot be avoided.

A Virtual Prototype (VP) of an HW platform can bridge the gap between HW and SW. It can merge both development phases, resulting in a better time-to-market and earlier verification and testing phases. Fig. 1.1 shows the general idea behind VP utilization in digital system development.

1.2 EDduino Virtual Platform

Our chair maintains and utilizes the open-source Extendable Translating Instruction Set Simulator (ETISS) [9, 10], which can be used to build virtual prototypes of various architectures. Currently, OpenRISC and RISC-V architectures are supported. With an extending library, the ETISS-SC Library [11], ETISS can easily interface with SystemC components, which helps build VPs of more complex System on Chips (SoCs). ETISS takes the part of a Central Processing Unit (CPU).

1.2.1 System on Chip

We virtualized a simple imaginary SoC, the EDduino, for this semester's laboratory part. Fig. 1.2 shows the SoC's block diagram.

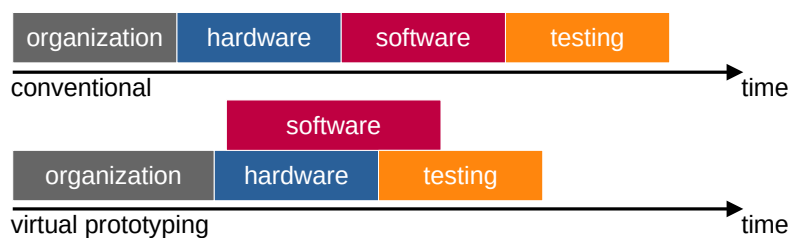


Fig. 1.1: Acceleration of digital system development through Virtual Prototypes

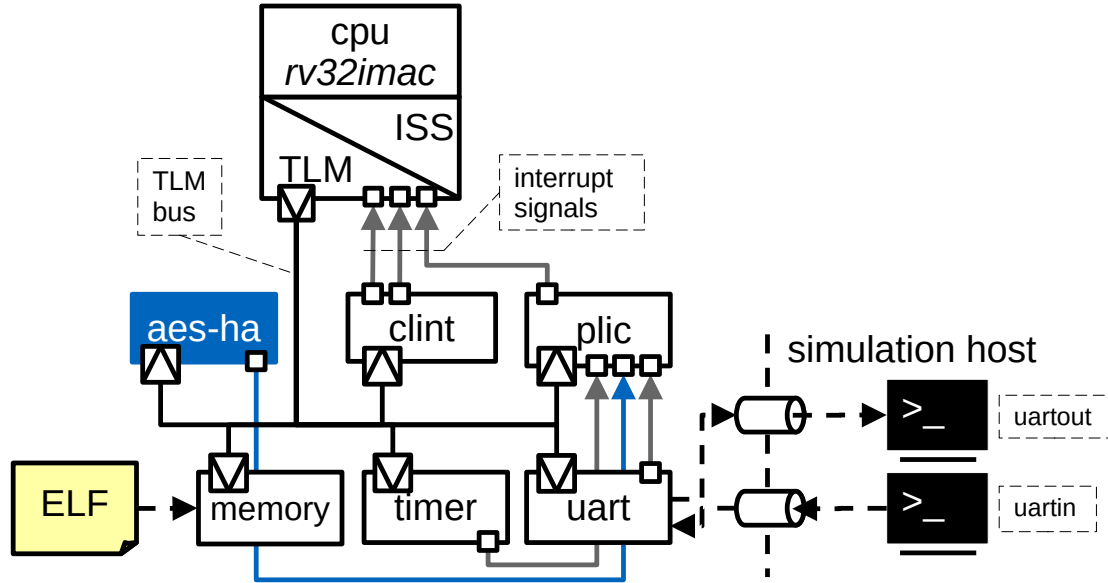


Fig. 1.2: EDAdino - System on Chip and Virtual Platform

CPU

The CPU core implements the 32-bit RISC-V Integer, Multiplier, Atomic, Compressed (IMAC [5]) and privileged instruction set ([7]). In the EDAdino-related laboratory parts, we will avoid writing any assembly code. Thus, the Instruction Set Architecture is not of much concern. Further interest in RISC-V can be satisfied by inspecting the various open-source specifications and tools. There is not one "RISC-V core" but many cores, sometimes proprietary, that implement the open RISC-V Instruction Set Architecture (ISA).

SoC Bus

The simplified SoC bus interconnects EDAdino's various components. The bus is implemented on Transaction-level Modeling (TLM) abstraction without any specific bus protocol. The bus routes memory-mapped accesses of the CPU to memory and peripherals. All components are memory-mapped, which means the CPU reads and writes (R/W) peripheral registers equally to the main memory through its Load and Store Unit (LSU) implemented in the core.

Interrupt Lines

Besides read/write (R/W) accesses initiated by the CPU, events in the form of interrupts from peripherals to the CPU can also be utilized. If applied, this can allow better resource

utilization than a polling-based software design, where software periodically reads and interprets a peripheral state held in a register.

Memory

The main memory contains both data and instruction memory. A linker script provided by the Software Development Kit (SDK) will map our binary in Executable and Linkable Format (ELF) to the correct memory addresses. The memory is initialized with this binary, which enables the CPU to execute the cross-compiled target software, provided it fetches from the valid start address, thus having the valid program entry.

CLINT

The Core-local Interrupt Controller (CLINT) is a SoC component that supplies a RISC-V core with two different interrupts. Multi-core RISC-V setups can use the Machine Software Interrupt (MSI) for inter-core signaling. The second interrupt provides the core with a fixed Real-Time Clock (RTC) sourced Machine Timer Interrupt (MTI). The software can use this interrupt as a time base for various applications, e.g., as a system tick for Real-Time Operating System (RTOS).

PLIC

The Platform-level Interrupt Controller (PLIC) is a memory-mapped interrupt controller that multiplexes input platform interrupt lines to the CPU core. In RISC-V machine privilege mode, only one interrupt line is specified for external interrupts, aka peripheral interrupts. To allow multiple SoC components, such as the Universal Asynchronous Receiver/Transmitter (UART) and the AES-HA, to use one single interrupt line to the CPU, the PLIC can be configured to prioritize these interrupt lines. Once one or more PLIC input lines are high (level-triggered) or rising high (edge-triggered), the PLIC output line to the CPU is set, and the active input interrupt with the highest priority is stored in a PLIC Claim/Complete-Register (CCR). The CPU can now claim the interrupt by reading the CCR, handling it, and completing the interrupt routine by writing to the CCR. The CPU can claim/complete multiple pending interrupts simultaneously.

UART

As a communication interface, EDAdino provides a mod. 16750 UART Peripheral. We can open a bi-directional communication interface to the simulation's host system to simulate an external device connected to the VP. This allows the simulation environment, i.e., another Linux terminal, to interface with EDAdino.

AES HA

The Advanced Encryption Standard (AES) Hardware Accelerator (HA) will be used as an example use case of High-Level Synthesis (HLS). During the laboratory, we will use various tools to translate a high-level input source (C/C++) to an HA integrated into the EDAduino SoC.

1.3 Toolchain

The VP setup can be identified as two separate build processes. The VP's branch utilizes ETISS and ETISS-VP libraries, as well as EDAduino SoC sources. In a later stage, EDAduino will also instantiate and connect the AES-HA to the SoC bus, thereby providing the target software with memory-mapped access to the accelerator. The target software's branch differs from the VP branch in that it is compiled by a cross compiler for its target ISA (RISC-V 32-bit), while the VP runs on our x86 host laboratory Linux machines. The bare-metal application software can use provided driver sources (board-/chip support package). Fig. 1.3 illustrates these two build branches. Both use CMake [8] as a build management tool to ease development.

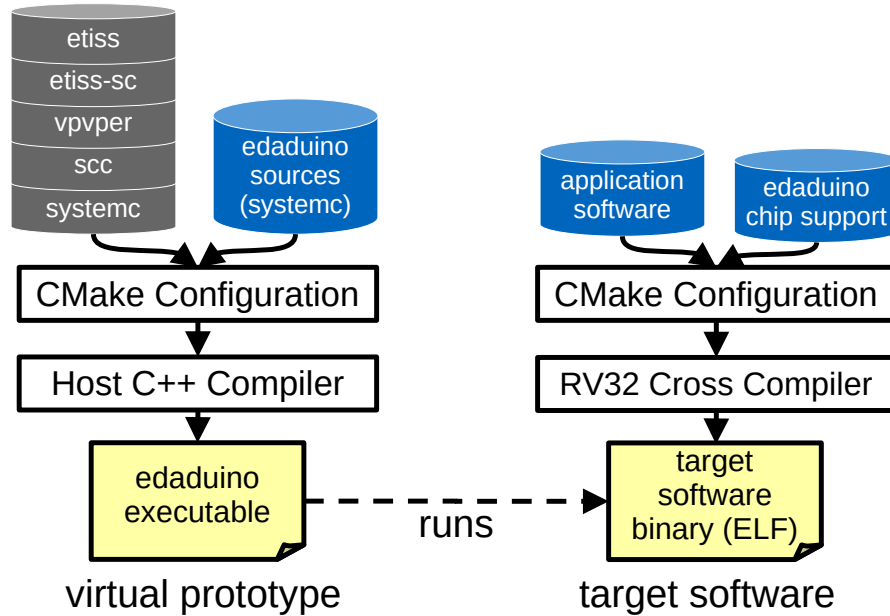


Fig. 1.3: Toolchain and build process

etiss [10], etiss-sc [11], vpvper [12], scc [13], systemc [2], cmake [8]

1.4 Hello World! Tutorial

This tutorial is intended as a walk-through for basic usage of the EDAdduino VP. To ease maintenance, we suggest working with the environment and tools provided in this semester's laboratory server directory. The paths used in this tutorial are intended to be used in your home directory `/homes/g{student.matricelnumber}`, where you can clone each lab module via git clone commands.

1.4.1 Getting Started

In general, you should work on your personal directory, as some scripts might assume they are executed within your home directory context.

1. Change into your personal working directory:

```
cd $HOME
```

2. Clone lab-module (I hope this works for you; otherwise, try SSH)| The VM already has a folder for all 3 modules:

```
git clone https://gitlab.tuwien.ac.at/hls/aca_ws25/lab-module-1.git
```

3. Execute the setup script (still in your home directory!!!). This sets up the required environment and patches some configuration files to your needs:

```
./lab-module-1/setup.sh
```

Now, you have a personal copy of the project files for the first module. In the VP sub-directory, we will compile the virtual prototype (`lab-module-1/vp`). In the SW sub-directory, we will cross-compile the target software (`lab-module-1/sw`). Inside these directories, README.md files also explain the build process. However, for this lab, we will go through them in this manual. For every new terminal, you should source the EDAdduino/lab-module environment:

0. Source the environment:

```
source $HOME/lab-module-1/environment.sh
```

Build the VP

0. As usual, source the environment:

```
source $HOME/lab-module-1/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-1/vp
```


1 Lab Module 1: Virtual Prototyping

2. Configure your VP-build in a **build** directory:

```
cmake -S $VP_DIR -B build
```

3. Build the VP with Make (via CMake):

```
cmake --build build
```

Together with some initial configuration files passed as command line arguments, the binary **build/edarduino** represents the executable VP. However, without a cross-compiled target software, we will run into issues with the startup of the VP.

Build the Target Software

0. As usual, source the environment:

```
source $HOME/lab-module-1/environment.sh
```

1. Change into the target software directory:

```
cd $HOME/lab-module-1/sw
```

2. Configure your SW-build in a **build** directory:

```
cmake -S . -B build
```

3. Build the target software with Make (via CMake):

```
cmake --build build
```

The cross-compiled binary/ELF **bm.elf** can be found in the build directory. Each time you make changes to your application software (in **main_bm.c**), you have to rebuild your binary (step 3). If you add a new source, you need to adapt the **CMakeLists.txt** and invoke **cmake** (step 2) again. This is optional. All necessary files are provided.

Run Target Software on VP

There are different ways to use and debug the VP. The target software could be ran freely (A)) or debugged on the VP via the ETISS GDB server (B)). The latter can be compared to debugging an embedded software via a JTAG or SWD interface on a physical development board that connects to the SoC's debug module.

A) Run Free:

0. As usual, source the environment:

```
source $HOME/lab-module-1/environment.sh
```

1 Lab Module 1: Virtual Prototyping

1. Change into the VP directory:

```
cd $HOME/lab-module-1/vp
```

2. Start the VP via the command line. The command line arguments `-etiss` and `-vp` specify configurations for ETISS and EDAdino, respectively. With `-elfs` the ELF-loader of our VP gets told which ELF-files should be flashed into the VP's memory structure:

```
build/edduino --etiss ini/etiss.ini --vp ini/vp.ini --elfs ../sw/build/bm.elf
```

- (optional) Open a second terminal, change into VP directory, and listen to the UART:

```
cd $HOME/lab-module-1/vp
```

```
cat uartout
```

- (optional) Open a third terminal, change into VP directory, and open the UART input FIFO for write:

```
cd $HOME/lab-module-1/vp
```

```
cat > uartin
```

Fig. 1.4 shows three terminal sessions. On the left-hand side, the free-run VP executes the provided HelloWorld target software and poll-based UART mirror. On the right-hand side, a second terminal running `cat` on the FIFO output file opened by the UART SystemC socket, and a third terminal waiting for input to the UART input FIFO. Note the two different printouts - 'HelloWorld! From ETISS!' and 'HelloWorld! From stdio'. If we look into the target software's `main_bm.c` in listing 1.1, we can find the responsible expressions in line 84 and 85. `STDIO's printf` is routed to the UART through EDAdino low-level drivers while `custom_print_string` is forwarded to a special memory range reserved for ETISS debugging. Both can be used to evaluate the target software. The third terminal emulates an open serial line to the VP's UART port. Type in any text and press Enter; this will issue a write to the UART peripheral, whose line register is currently polled by the target software to wait for new data. This data should then be transmitted over the UART's output, resulting in a mirroring I/O. Note that although the application is in "busy wait" for UART input, the system timer still reports ticks. This is because the CLINT timer interrupts the application.

1 Lab Module 1: Virtual Prototyping

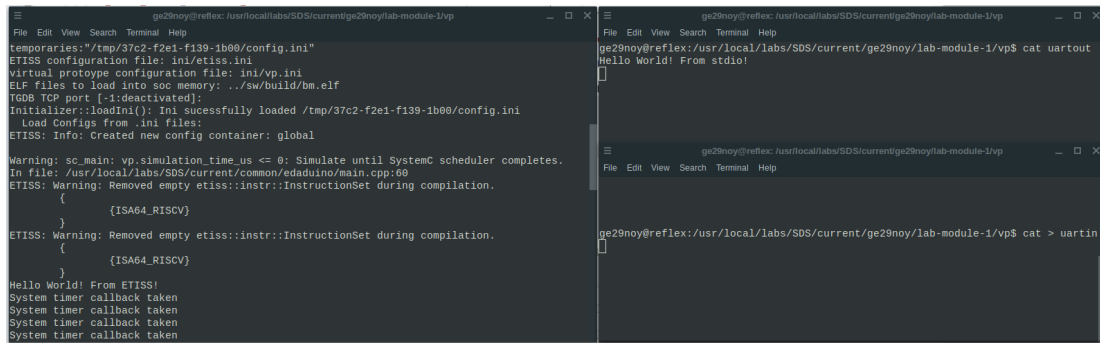


Fig. 1.4: Run the VP and target software freely (+ UART listen)

B) Target Debug:

0. As usual, source the environment:

```
source $HOME/lab-module-1/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-1/vp
```

2. Start the VP via the command line. With the `-tgdb <tcp-port>` command line parameter, the GDB server configuration is specified. The VP should now halt automatically on startup and wait for a remote GDB connection to the specified TCP port:

```
build/edaduino --etiss ini/etiss.ini --vp ini/vp.ini --elfs ../sw/build/bm.elf --tgdb 2022
```

- (optional) Open a second terminal, change into VP directory, and listen to the UART:

```
cd $HOME/lab-module-1/vp
```

```
cat uartout
```

- (optional) Open a third terminal, change into VP directory, and open the UART input FIFO for write:

```
cd $HOME/lab-module-1/vp
```

```
cat > uartin
```

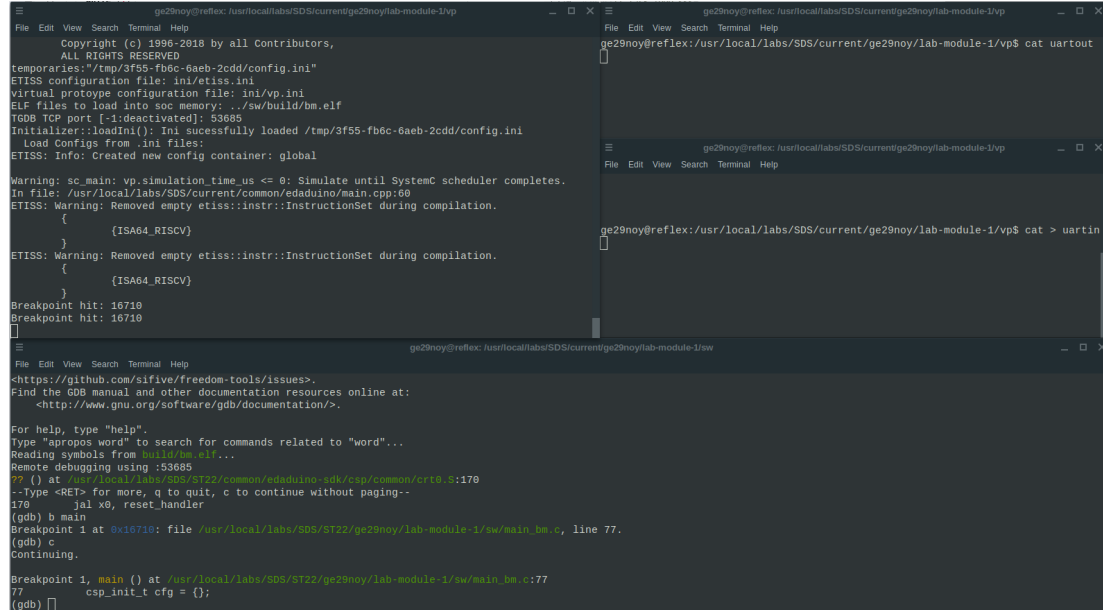
3. Open a fourth terminal, change into SW directory, and start a remote GDB session:

```
cd $HOME/lab-module-1/edaduino/sw
```

1 Lab Module 1: Virtual Prototyping

```
$RISCV/bin/riscv32-unknown-elf-gdb -ex "tar rem :2022" \
    build/bm.elf
```

Fig. 1.5 shows the terminal setup for target debugging. While mastering a Unix command line is certainly challenging and nice to have on a CV, even working with a new tool that supplies a graphical user interface can be hard enough. Until now, all steps could be done with the command line only; in the next section 1.5, we will integrate these steps to build a small graphical software development kit.



```
Copyright (c) 1996-2018 by all Contributors,
ALL RIGHTS RESERVED
temporaries: /tmp/3f55-fb6c-6aeb-2cdd/config.ini
ETISS configuration file: ini/etiss.ini
virtual prototype configuration file: ini/vp.ini
ELF files to load into soc memory: ../sw/build/bm.elf
TGD TCP port [-1:deactivated]: 53685
Initializer: loadini(): ini successfully loaded /tmp/3f55-fb6c-6aeb-2cdd/config.ini
Load Configs from .ini files:
ETISS: Info: Created new config container: global

Warning: sc_main: vp.simulation_time_us <= 0: Simulate until SystemC scheduler completes.
In file: /usr/local/labs/SDS/current/common/edaduino/main.cpp:60
ETISS: Warning: Removed empty etiss::instr::InstructionSet during compilation.
{
  {ISA64_RISCV}
}
ETISS: Warning: Removed empty etiss::instr::InstructionSet during compilation.
{
  {ISA64_RISCV}
}
Breakpoint hit: 10710
Breakpoint hit: 10710

<https://github.com/sifive/freedom-tools/issues>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/bm.elf...
Remote debugging using :53685
77 () at /usr/local/labs/SDS/ST22/common/edaduino-sd/csp/common/crt0.S:170
..Type <RET> for more, q to quit, c to continue without paging..
170      jal x0, reset_handler
(gdb) b main
Breakpoint 1 at 0x10710: file /usr/local/labs/SDS/ST22/ge29noy/lab-module-1/sw/main_bm.c, line 77.
(gdb) c
Continuing.
Breakpoint 1, main () at /usr/local/labs/SDS/ST22/ge29noy/lab-module-1/sw/main_bm.c:77
77      csp_init_t cfg = {};
(gdb)
```

Fig. 1.5: Run the VP and target software in target debug mode (+ UART)

1.5 IDE

The following steps are tailored for this laboratory's purposes, which include hard-coded paths in the VS Code workspace settings for each lab module folder. Therefore, this SDK might not work outside the <acalab> environment.

- C/C++ Extension and Native Debug is recommended for the lab.
- If IntelliSense in VS Code is not working correctly for the C/C++ Extension, you can copy the *c_cpp_properties.json* file and rename it to *settings.json* inside the same *.vscode* folder.

0. As usual, source the environment:

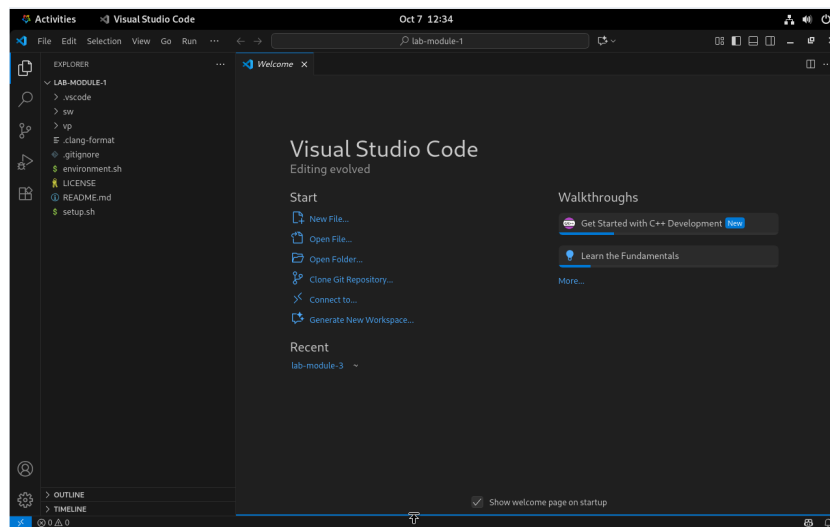
```
source $HOME/lab-module-1/environment.sh
```

1. Start VS Code:

```
code lab-module-1
```

2. (Optional) Open VS Code Terminal by dragging up the bottom: Short-cut

```
Ctrl + `
```

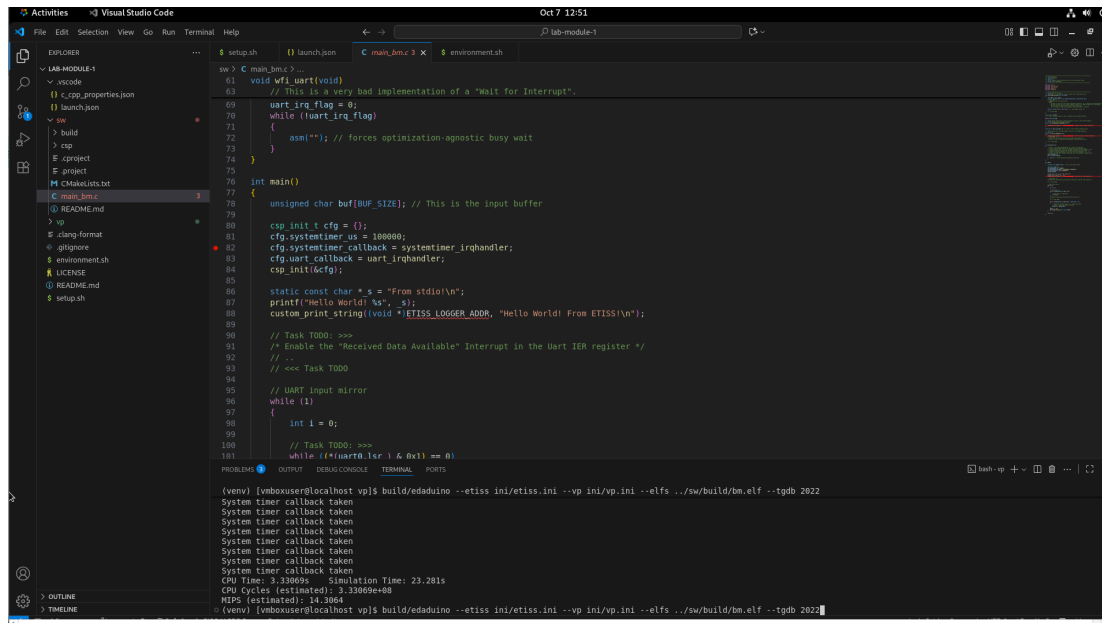


3. Open a separate terminal or use the VS Code one, source environment, change directory to VP, launch the VP with remote configuration:

```
cd $HOME/lab-module-1/edaduino/vp
```

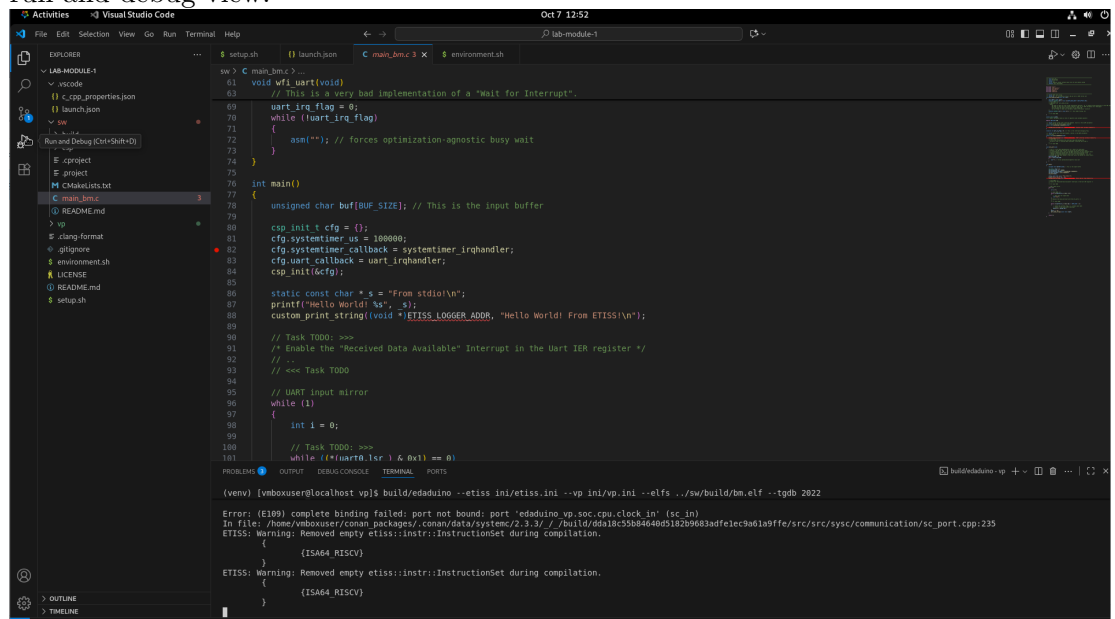
```
build/edaduino --etiss ini/etiss.ini --vp ini/vp.ini --elfs ../sw/build/bm.elf --tgdb 2022
```

1 Lab Module 1: Virtual Prototyping



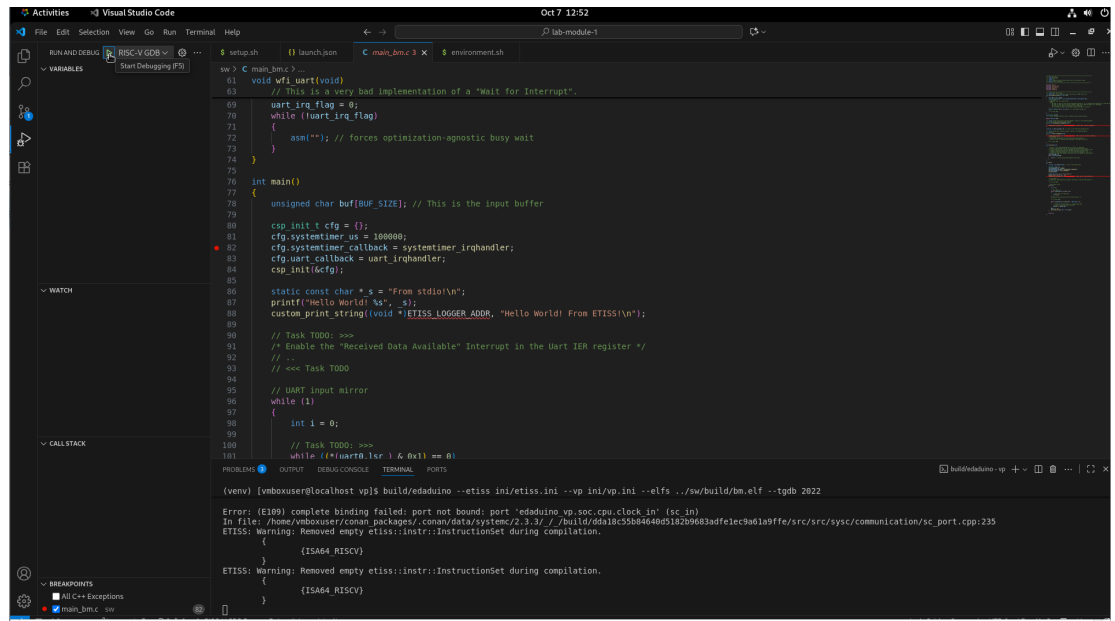
4. Launch the pre-configured remote debug session in your VS Code File (Requires Native Debug Extension):

- First time: Set a breakpoint into an interesting function and click on to the run and debug view.



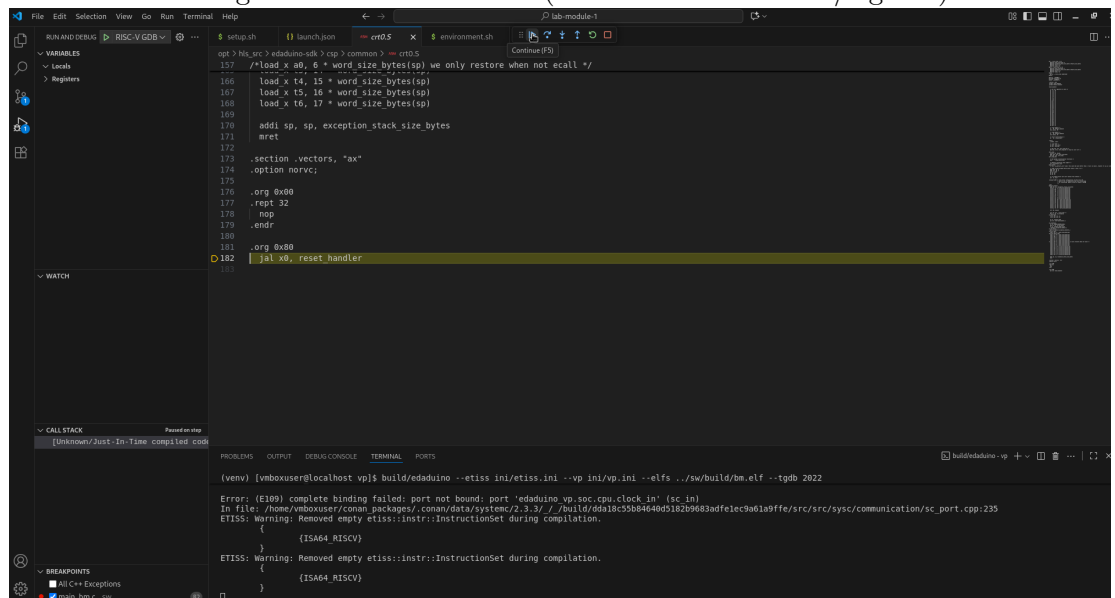
- Launch the debug session. You can choose between multiple in lab 3. (F5 also works to start currently selected one)

1 Lab Module 1: Virtual Prototyping



```
61 void wfi_uart(void)
62 {
63     // This is a very bad implementation of a "Wait for Interrupt".
64     while (uart_irq_flag == 0)
65     {
66         asm(""); // forces optimization-agnostic busy wait
67     }
68 }
69
70 int main()
71 {
72     unsigned char buf[BUF_SIZE]; // This is the input buffer
73
74     csp_init + cfg = 0;
75     csp_systemtimer_us = 100000;
76     csp_systemtimer_callback = systemtimer_irqhandler;
77     csp_uart_callback = uart_irqhandler;
78     csp_init(&cfg);
79
80     static const char * s = "From stdio\n";
81     printf("Hello World! %s", s);
82     custom_print_string(void *ETISS_LOGGER_ADDR, "Hello World! From ETISS\n");
83
84     // Task TODO: >>>
85     // Enable the "Received Data Available" Interrupt in the UART IER register */
86     // <<< Task TODO
87
88     // UART input mirror
89     while (1)
90     {
91         int i = 0;
92
93         // Task TODO: >>>
94         while ((uart_irq_flag) & 0x1 == 0)
95         {
96             // Task TODO: >>>
97             while ((uart_irq_flag) & 0x1 == 0)
98             {
99                 // Task TODO: >>>
100                 while ((uart_irq_flag) & 0x1 == 0)
101                 {
102                     // Task TODO: >>>
103                     while ((uart_irq_flag) & 0x1 == 0)
104                     {
105                         // Task TODO: >>>
106                         while ((uart_irq_flag) & 0x1 == 0)
107                         {
108                             // Task TODO: >>>
109                             while ((uart_irq_flag) & 0x1 == 0)
110                             {
111                                 // Task TODO: >>>
112                                 while ((uart_irq_flag) & 0x1 == 0)
113                                 {
114                                     // Task TODO: >>>
115                                     while ((uart_irq_flag) & 0x1 == 0)
116                                     {
117                                         // Task TODO: >>>
118                                         while ((uart_irq_flag) & 0x1 == 0)
119                                         {
120                                             // Task TODO: >>>
121                                             while ((uart_irq_flag) & 0x1 == 0)
122                                             {
123                                                 // Task TODO: >>>
124                                                 while ((uart_irq_flag) & 0x1 == 0)
125                                                 {
126                                                     // Task TODO: >>>
127                                                     while ((uart_irq_flag) & 0x1 == 0)
128                                                     {
129                                                         // Task TODO: >>>
130                                                         while ((uart_irq_flag) & 0x1 == 0)
131                                                         {
132                                                             // Task TODO: >>>
133                                                             while ((uart_irq_flag) & 0x1 == 0)
134                                                             {
135                                                                 // Task TODO: >>>
136                                                                 while ((uart_irq_flag) & 0x1 == 0)
137                                                                 {
138                                                                     // Task TODO: >>>
139                                                                     while ((uart_irq_flag) & 0x1 == 0)
140                                                                     {
141                                                                         // Task TODO: >>>
142                                                                         while ((uart_irq_flag) & 0x1 == 0)
143                                                                         {
144                                                                             // Task TODO: >>>
145                                                                             while ((uart_irq_flag) & 0x1 == 0)
146                                                                             {
147                                                                                 // Task TODO: >>>
148                                                                                 while ((uart_irq_flag) & 0x1 == 0)
149                                                                                 {
150                                                                                     // Task TODO: >>>
151                                                                                     while ((uart_irq_flag) & 0x1 == 0)
152                                                                                     {
153                                                                                         // Task TODO: >>>
154                                                                                         while ((uart_irq_flag) & 0x1 == 0)
155                                                                                         {
156                                                                                             // Task TODO: >>>
157                                                                                             while ((uart_irq_flag) & 0x1 == 0)
158                                                                                             {
159                                                                                                 // Task TODO: >>>
160                                                                                                 while ((uart_irq_flag) & 0x1 == 0)
161                                                                                                 {
162                                                                                                     // Task TODO: >>>
163                                                                                                     while ((uart_irq_flag) & 0x1 == 0)
164                                                                                                     {
165                                                                                                         // Task TODO: >>>
166                                                                                                         while ((uart_irq_flag) & 0x1 == 0)
167                                                                                                         {
168                                                                                                             // Task TODO: >>>
169                                                                                                             while ((uart_irq_flag) & 0x1 == 0)
170                                                                                                             {
171                                                                                                                 // Task TODO: >>>
172                                                                                                                 while ((uart_irq_flag) & 0x1 == 0)
173                                                                                                                 {
174                                                                                                                     // Task TODO: >>>
175                                                                                                                     while ((uart_irq_flag) & 0x1 == 0)
176                                                                                                                     {
177                                                                                                                         // Task TODO: >>>
178                                                                                                                         while ((uart_irq_flag) & 0x1 == 0)
179                                                                                                                         {
180                                                                                                                             // Task TODO: >>>
181                                                                                                                             while ((uart_irq_flag) & 0x1 == 0)
182                                                                                                                             {
183                                                                                         
```

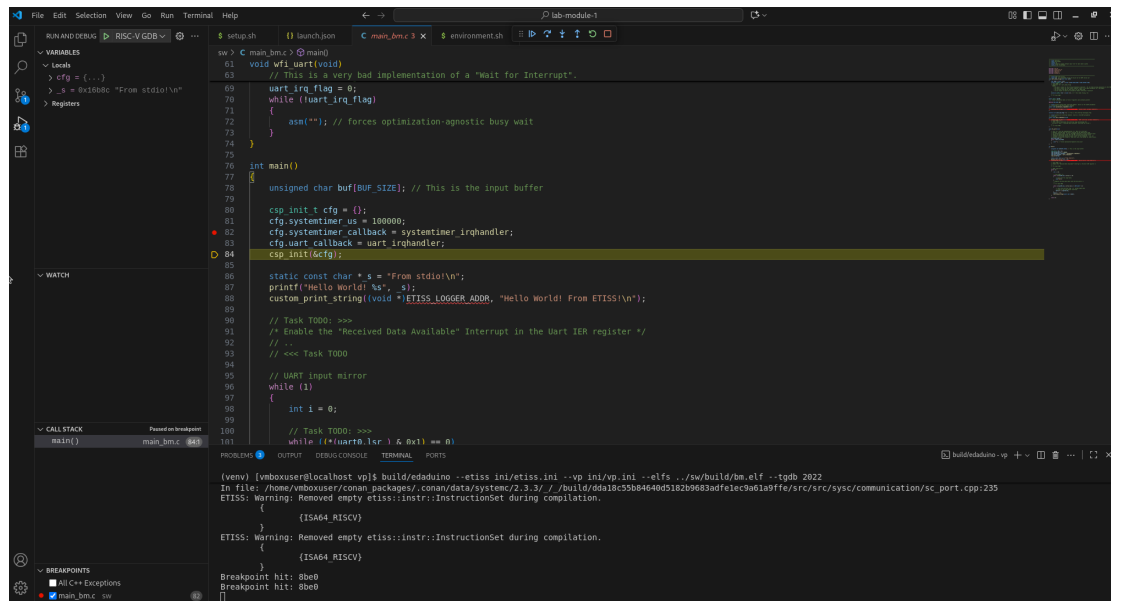
- Continue once to get out of reset handler (Bare Metal start code/ Ignore)



```
157 .opt > No,src > edaduno-ide > csp > common > crt.s
158
159 /*load x a0, 6 * word_size_bytes(sp) we only restore when not ecall */
160 load x t4, 15 * word_size_bytes(sp)
161 load x t5, 16 * word_size_bytes(sp)
162 load x t6, 17 * word_size_bytes(sp)
163
164 addi sp, sp, exception_stack_size_bytes
165
166 mret
167
168 .section .vectors, "ax"
169 .option norvc;
170
171 .org 0x00
172 .rept 32
173 nop
174 .endr
175
176 .org 0x80
177 jal x0, reset_handler
178
179 
```

- Finally at the breakpoint.

1 Lab Module 1: Virtual Prototyping



1.6 Task 1: Driving UART on a Bare-metal System

The embedded system's task is to mirror its serial input on its serial output. To determine when new data is available in the UART's Receiver Buffer Register (RBR), the UART's Line Status Register (LSR) is continuously read, specifically its least significant bit (LSR.LSB). This bit indicates the receiver overrun state, i.e., new data is available. This synchronization approach is also known as "polling". Polling has advantages and disadvantages. On the plus side, polling is straightforward and relatively predictable. On the other hand, it could be quite inefficient if the sampling period is not well adjusted to the system's behavior. For example, to minimize the reaction time to a new input in the UART, we need to read the LSR as frequently as possible. If we now read the LSR at any possible moment, we can not utilize the CPU for another task. In contrast, polling the LSR every other 1 millisecond or so would mean possibly losing input data due to a more frequently updating UART device.

Your task for this lab part is to modify `sw/main_bm.c` accordingly:

- i) complete the `uart_send_string` function definition
- ii) replace the "busy poll-wait" with an interrupt-based synchronization

1.6.1 Complete UART Send String Function

The `uart_send_string(const char *str)` (L. 18-33 in Listing 1.1) function is incomplete. The UART's LSR can be read to determine the status of the Transmitter Holding Register (THR). A byte-write to the THR will result in an output to the UART's serial line, however, we have to make sure that "THR is empty" before writing new data to it, which is the case if. This is the case, if the bit number 5 of the 8-bit LSR is set. The `volatile` keyword hints the compiler that source code accessing the associated variable must not be optimized away. By declaring the memory references `thr` and `lsr` volatile, the compiler will know not to optimize read and writes to this memory.

1.6.2 Wait for Interrupt

Replace the "busy poll-wait" (L. 100-108 in Listing 1.1) with a call to `wfi_uart()` which emulates a function yielding the processing unit to the operating system's scheduler. In this lab, this is, ironically, simulated by yet another "busy poll-wait" on the global `uart_irq_flag`.

1.7 Questions (for practice only)

1. What are memory-mapped peripherals?
2. What is the reason of declaring register references as volatile?
3. What are the contents of `Uart/uart_t` struct in the UART driver library.
4. What is busy waiting?
5. Is polling, in general, a bad design choice? Discuss.
6. Why is there still a system tick reported, while the processing system is in busy wait state? (Bare-metal system)
7. What is cross compilation?

1.8 Appendix

1.8.1 Task 1:

```
1 ///////////////////////////////////////////////////////////////////
2 /// \file main_bm.c
3 /// \date 04/23/2020
4 /// \author JG
5 /// \brief edarduino target software main file for bare metal system
6 /// \details mod. 04/25/2022
7 ///////////////////////////////////////////////////////////////////
8
9 #include "csp.h"
10 #include "cust_print.h"
11 #include "uart_drv.h"
12 #include <stdio.h>
13 #include <stdlib.h>
14
15 ///////////////////////////////////////////////////////////////////
16 /// \brief UART send string
17 /// \param str '\0' terminated string to be put out on UART serial out
18 void uart_send_string(const char *str)
19 {
20     char *buf = (char *)str;
21     volatile uint8_t *thr = (void *) (UART_BASE_ADDR + UART_OFFSET_THR);
22     // Task TODO: >>>
23     /* Implement the 'uart_send_string'
24     HINTS:
25         - We need to check for the string termination character '\0' to break writing
26         characters to the THR register
27         - We need to check the line status register (LSR) of our UART peripheral for "THR
28         empty"
29         - You can poll the LSR to "busy wait" for an empty THR
30         - do not forget to make your peripheral memory references "volatile"
31     */
32     volatile uint8_t *lsr = (void *)-1; //<-- this needs fixing, too
33     // ...
34     // <<< Task TODO
35 }
36
37 extern uart_t uart0;
```

1 Lab Module 1: Virtual Prototyping

```
36 ///< 'holds' peripheral data in form of registers and callback pointers
37
38 #define BUF_SIZE 100
39
40 ///
```

```

98     int i = 0;
99
100     // Task TODO: >>>
101     while ((*uart0.lsr_) & 0x1) == 0)
102     {
103         // busy wait for input data
104         asm("nop");
105     }
106     /* Replace the busy poll-wait with the wfi_uart() */
107     // ..
108     // <<< Task TODO
109
110     while ((*uart0.lsr_) & 0x1) && i < (BUF_SIZE - 1))
111     {
112         // check the overflow flag, i.e. unread input data
113         // break if the read buffer overflows
114         buf[i++] = *uart0.rbr_;
115     }
116     buf[i] = '\0';
117     uart_send_string((const char *)buf);
118 }
119
120 return 0;
121 }

```

Listing 1.1: Helloworld tutorial bare-metal target software main_bm.c

1.9 FAQ

- Q: When doing the lab tasks, do we have to change the toolchain settings, e.g., the CMakeFiles for adding additional programming sources?
A: The lab is designed so that you can do all the relevant stuff in the main source files `edaduino/sw/main_bm.c`. Thus, there is no need to make any changes to the provided development environment. 1
- Q: In case we want to add new sources to the project. How?
A: You can do that by adding the needed configurations to the relevant CMake file, e.g. `sw/CMakeLists.txt`. This requires some understanding of CMake, which we do not teach in our lab.
- Q: In the current task, we have to insert the Wait For Interrupt call into our code. Do we have to make any changes to the function itself?
A: No, all code sections where you need to change something in `edaduino/sw/-main_bm.c` are labeled with the following comment bracket.

```

1 // Task TODO: >>>
2 // .. -> solve here
3 // <<< Task TODO

```

- Q: Did you already add all students to the lab list, since I can't access the TiLAB or my `g{student.matricelnumber}` folder?
A: All directories are created after the first laboratory introduction lecture. If you can not access your working directory, please contact a supervisor.

1 Lab Module 1: Virtual Prototyping

- Q: Wait, I cant access the `tuwien.gitlab` project sources?
A: You have to log in once to the `GitLab.tuwien` instance. I can only add you afterwards, unfortunately. I will do this automatically with a script. Please ask me for access if I haven't invited you automatically.

2 Lab Module 2: High Level Synthesis with Vivado-HLS

In this lab module, you will be introduced to the Vivado-HLS tool to transform your C-level functionality directly into an HDL design. This HDL design can then be used as a packaged-IP in the PL part of the Zynq chip by importing it into the Vivado *IP-Catalog*. Specifically, we will follow the following tutorials from the Xilinx Vivado-HLS tutorial [15] as part of this lab task. Please download the tutorial PDF from the Moodle page.

2.0.1 Preparation

1. Change into your personal working directory:

```
cd $HOME
```

2. Clone lab-module (I hope this works; otherwise, try SSH):

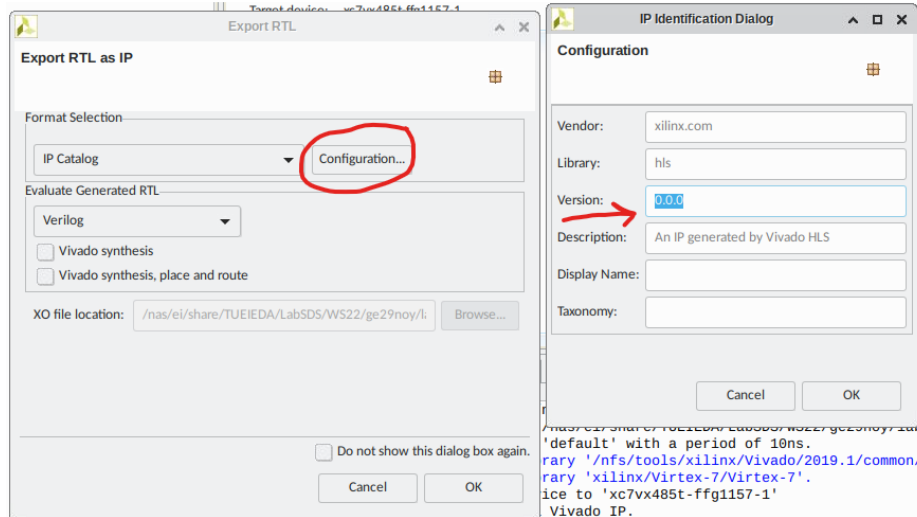
```
git clone https://gitlab.tuwien.ac.at/hls/aca_ws25/lab-module-2.git
```

2.0.2 Part 1 - Introduction

1. Change into your working directory and the project directory for this task. The files needed for this task are located here.

```
cd $HOME/lab-module-2/p1_getting_started
```

3. Do the tutorial *Ch. 2: Lab1: Creating a High-Level Synthesis Project* from *Xilinx Tutorial Manual (UG871)* and adapt accordingly as per your working-directories. This tutorial will teach you the basics about the Vivado-HLS tool and its GUI interface. The necessary project files have been copied to your working directory in the previous step. Make sure to place your Vivado-HLS project in this directory. **Note:** Should there be an error message when packaging/exporting the IP, try setting the value for the *Version* parameter to 0.0.0 in the *Export RTL as IP dialog*:



2.0.3 Part 2 - Design Analysis

1. Change into your working directory and the project directory for this task. The files needed for this task are located here.

```
cd $HOME/lab-module-2/p2_design_analysis
```

2. Do the tutorial *Ch. 6: Design Analysis* from *Xilinx Tutorial Manual (UG871)* and adapt accordingly as to your working directory. Ensure that you place your Vivado-HLS project in this directory.

2.1 Questions (for practice only)

1. What is C-code validation and C-code synthesis in Vivado-HLS?
2. What is *Local* vs *Global* optimization? [Ref. to SDS Theory]
3. What are *Directives* in Vivado-HLS? List the available directives in Vivado HLS and provide a one-line description for each.
4. How does *loop-unrolling* works in optimizing the design?
5. What is the difference between *DATAFLOW* and function-level *PIPELINE* directives?
6. What is the difference between *LOOP_FLATTEN* and *UNROLL* directive?
7. What were the indicators that led to the conclusion that the initial design was not pipelined in *Step 3* of the DCT-HLS tutorial?
8. Why didn't we apply *PIPELINE*-directive directly to the `dct_1d` in *Step 4* of DCT-HLS tutorial?

9. What were the indicators that led to the conclusion that an imperfect loop nest was blocking the loop-pipelining in *Step 5* of the DCT-HLS tutorial?
10. What were the indicators that led to the conclusion that data-dependency was "bottlenecking" the design in *Step 6* of the DCT-HLS tutorial?
11. Describe briefly how an HDL design from Vivado-HLS can be exported to Vivado's *IP-Catalog*.

2.2 Appendix

2.2.1 FAQ

- Q: If there are so many restrictions on the code, and its used language features (memory, recursion,...) input to HLS, are there languages that are more suited for HLS?

Most importantly, do not interchange input languages with High Level Synthesis (HLS) with Hardware description languages. The key idea is to synthesize stuff not described in an HDL (e.g, register transfer logic in VHDL, Verilog, Chisel) into a hardware description. There are languages that might be more suited than others for HLS, such as function-oriented languages; however, we want to reuse already established code bases. We are only working with C and C++ inputs.

2.2.2 Reference Values

Table 2.1: Part 1) HLS Intro Tutorial (FIR)

solution	latency [clock cycles]	interval [clock cycles]
1	34	34

Table 2.2: Part 2) Design Analysis Tutorial (DCT_HLS)

solution	latency [clock cycles]	interval [clock cycles]
1	2935	2935
2	1723	1723
3	843	843
4	477	477
5	476	343
6	463	98

3 Lab Module 3: Advanced Encryption Standard

In this lab, we will use the Advanced Encryption Standard (AES) as a case study for application profiling. This means we will first explore how a pure software implementation of an AES-based encryption/decryption algorithm performs on EDAduino. We will see that the used library constitutes a significant performance bottleneck for the relatively small microcontroller-sized SoC. Next, we will explore how we can offload the CPU by introducing an HLS-generated hardware accelerator (HA). The HA's performance will be optimized by inserting HLS directives while keeping the resource usage (area) low. In addition to offloading the computational task to dedicated HW, this will allow the SoC to run the HA on a smaller clock frequency, thus reducing overall power consumption.

Although security is an important topic, we are not focusing on it in this High-Level Synthesis Laboratory. We will take some shortcuts, e.g., storing keys in source code, and will not develop a secure Crypto System on Chip. Off-the-shelf chips provide hardware-accelerated support for AES (See [6]).

3.0.1 AES Block Cipher

The block cipher AES operates on a fixed-length data block - in our case, 128 bits (16 bytes). Each block is symmetrically transformed (encryption and decryption use the same key) by the algorithm. The block diagram in Fig. 3.1 depicts the basic functionality of one such transformation. Each transformation first generates a set (Nr) of round-keys (**KeyExpansion**) from the input key since each round uses a different key. Next, the first round (1) performs a **AddRoundKey** operation on the AES state, which is a 4x4 Matrix representation of the input block. Rounds 2 to $Nr - 1$ sequentially perform **SubBytes**, **ShiftRows**, **MixColumns** and **AddRoundKey** functions on the state. At last, round Nr performs a **SubBytes**, **ShiftRows**, and a final **AddRoundKey** on the state. **SubBytes** is a non-linear function and can be implemented easily with the help of a look-up table, while the other functions use basic arithmetic. For AES-128 ($Nr = 10$), the overall computation is made out of 10 **AddRoundKeys**, 9 **SubBytes**, 9 **ShiftRows** and 8 **MixColumns**.

Depending on the processing unit performing the AES cipher and the message size (which may involve multiple blocks), this can lead to performance issues, making dedicated hardware more attractive.

3.0.2 Block Cipher Mode of Operation

Since AES operates on a fixed-sized block, to transform multiple blocks, e.g., a message, AES has to be invoked by a block cipher mode of operation (BCMO) that introduces

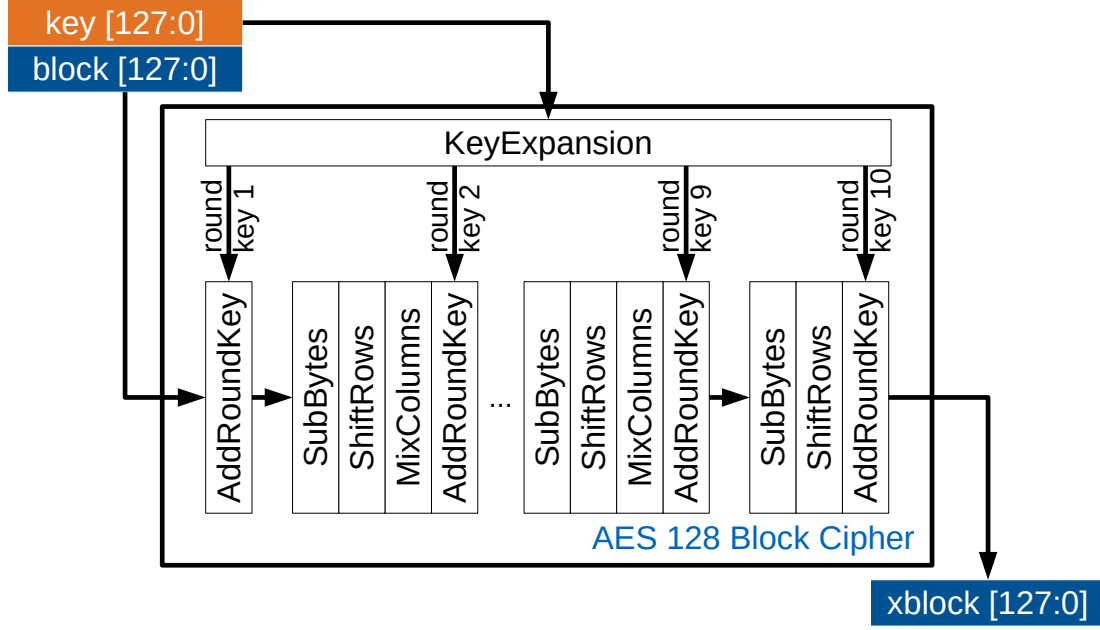


Fig. 3.1: AES-128 Block Cipher

an initialization vector (IV), allowing variations of the output for repeating elements in the input while performing the same block transformation on all elements. Various BCMOs exist; we will use the *Counter* (CTR) mode in this laboratory. CTR, as the AES block cipher, is symmetric (XOR of CTR is symmetric), thus allowing the implementing module to perform both encryption and decryption with the same computation. *Counter* refers to the IV being incremented with each input block. Fig. 3.2 shows the simplified *Counter* BCMO. Whether block{0 ... n} or xblock{0 ... n} refers to the encrypted or decrypted message makes no difference because the CTR AES can operate bidirectional due to its inherent symmetry. This allows software and, most importantly, hardware to use the same function for both cryptographic tasks.

3.0.3 AES Hardware Accelerator

The EDAdino SoC VP is based on SystemC, where most peripherals are modeled after defined protocols. For example, the UART peripheral implements a transaction-level model (TLM) of a 16750 protocol through memory-mapped registers. Since we want to use HLS, which yields a Register Transfer Level (RTL)-based IP, such as in VHDL or Verilog, integrating this IP within the EDAdino VP context is not as straightforward as implementing a protocol-conform behavior.

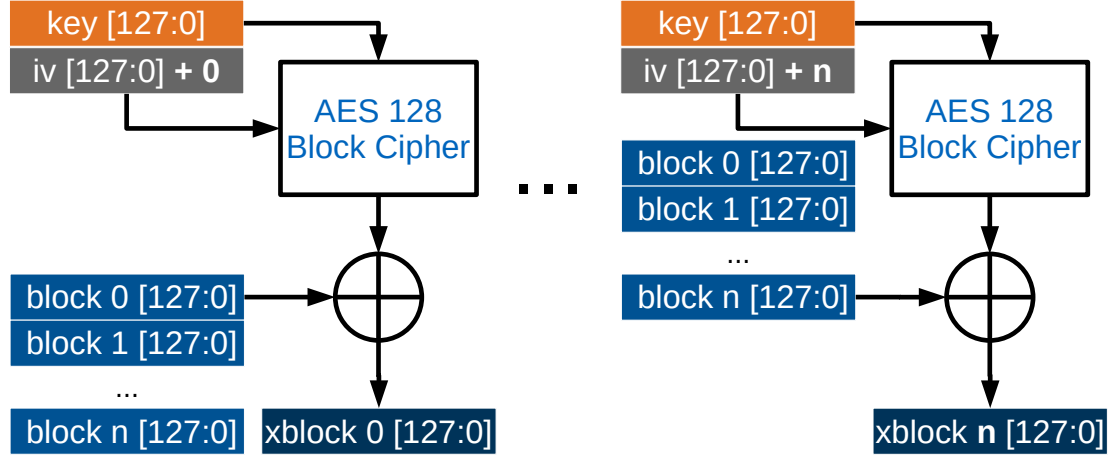


Fig. 3.2: Counter BCMO

Design Flow and Implementation

We will utilize a standard open-source tool to integrate the HA into the SoC: Verilator ([14]) transforms RTL models, specified in Verilog or SystemVerilog, to host-executable, cycle-accurate C++ or SystemC models (VRTL). Fig. 3.3 shows the basic design flow, which takes the high-level C function (the AES software library) and translates it to Verilog hardware description through Vivado-HLS. It then translates to a SystemC model, which EDAduino later instantiates. Inter-stage tests ensure that functionality is not lost between the different design stages. The design flow, except for the HLS optimization, is provided.

AES-HA Integration

Vivado HLS is primarily used to generate IP cores for Xilinx FPGAs. A so-called TLM Transactor manages the information flow between the AES-HA and the remaining VP to generate timing-dependent interface signals within the SystemC environment. The other VP components are not stimulated by clock cycles such that the transactor has to generate the HA clock. We have fixed the HLS top-level interface to synthesize as a *AMBA AXI4-Lite* [4] compatible module. This means all input and output parameters will be accessible to memory operations addressed with their respective offsets. The transactor consumes incoming memory transactions from its TLM target socket and generates the required *AMBA AXI4-Lite* signals for the HA's input ports. The TLM response data is constructed from the HA's output ports on memory reads. The *AMBA AXI4-Lite* target (the HA) can be programmed to trigger an interrupt on the standard events "Ready" and "Done." "Done" means one processed block is ready to be read. "Ready" means a new process may be started. This can be important if the HA allows pipelined execution. The CTR IV is stored in internal registers and is both readable and writable by the CPU for initialization and incremented by the AES-HA after each

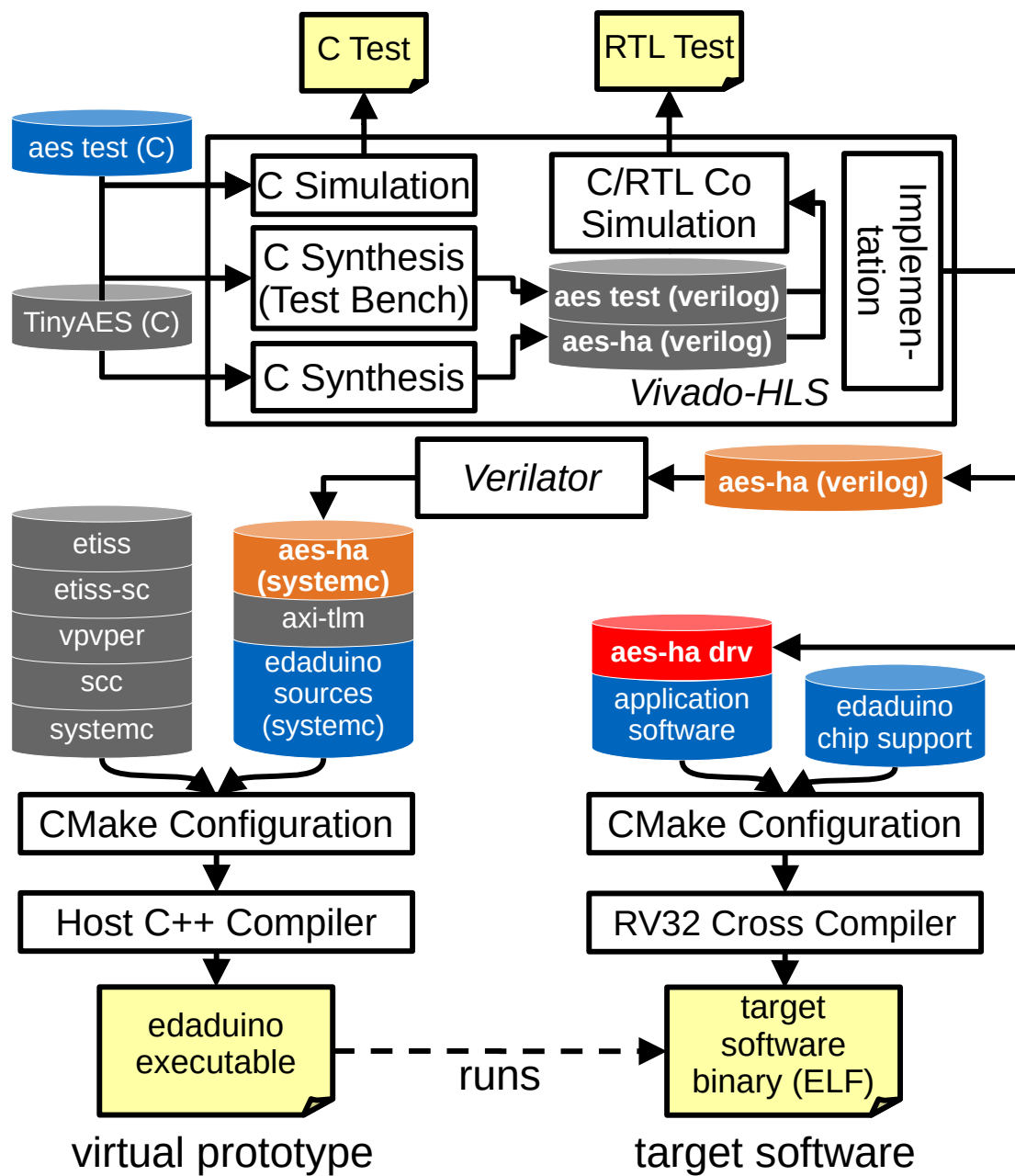


Fig. 3.3: Toolchain Flow. High Level (C) to RTL to C++/SystemC through Vivado-HLS and Verilator

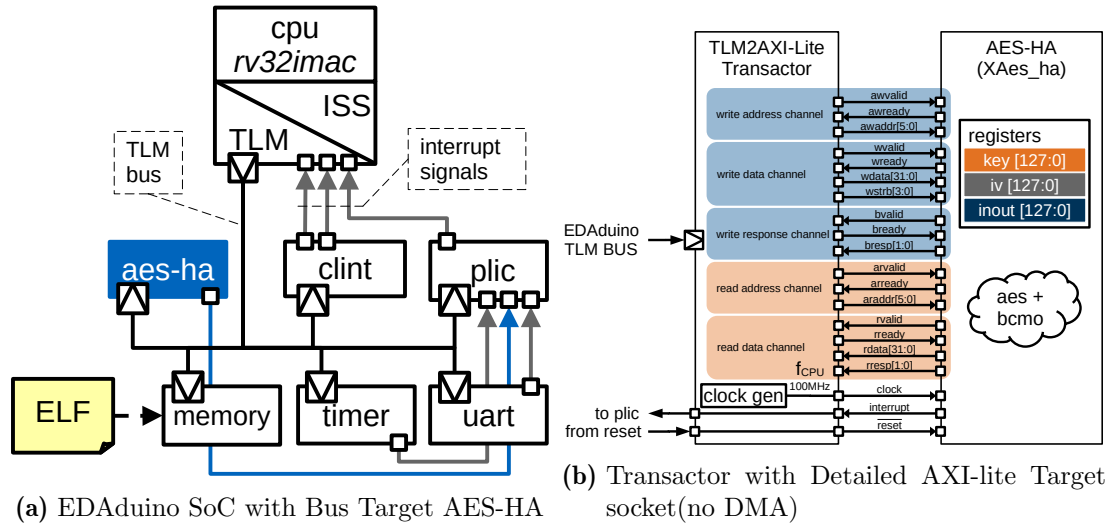


Fig. 3.4: Block model of pure bus target AES-HA and TLM-Transactor in EDAduino VP

128-bit run is completed. Fig. 3.4 depicts a block diagram of the HA integration as seen from the EDAduino bus.

The low-level software drivers for the standardized *AMBA AXI4-Lite* interface are automatically generated by Vivado-HLS. This abstracts the address based register control to comprehensible C-functions, e.g., `XAes_ha_Start` to start a process, to write the key bytes `XAes_ha_Write_key_Bytes`, to write the IV bytes `XAes_ha_Write_iv_Bytes`, `XAes_ha_Write_inout_r_Bytes` to write the IO registers, and `XAes_ha_Read_inout_r_Bytes` to read the IO registers.

3.0.4 Getting Started

Similar to the first laboratory module, this module relies on the tools provided by our server. We suggest working with the environment and tools provided in this semester's laboratory file directory to ease maintenance.

This directory (`/homes/g{student.matricelnumber}`) will be referred to as `$HOME`. Although not required, debugging software on the VP can be very effective when a GUI-based tool is used (see Module 1 for instructions on using VS Code with our VP).

In general, you should work in your personal `$HOME` directory, as some scripts might assume they are executed within the TILAB context.

1. Change into your personal working directory:

```
cd $HOME
```

2. Clone lab-module (I hope this works for you; otherwise, try SSH):

3 Lab Module 3: Advanced Encryption Standard

```
git clone https://gitlab.tuwien.ac.at/hls/aca_ws25/lab-module-3.git
```

3. Execute the setup script (still in your home directory!!!). This sets up the required environment and patches some configuration files to your needs:

```
./lab-module-3/setup.sh
```

Now, you have a personal copy of the VP directory (`lab-module-3/vp`), the target software (`lab-module-3/sw`), and the HLS project (`lab-module-3/hls`), which we use to generate the hardware accelerator. Inside these directories, README.md files also explain the build process. However, for this lab, we will go through them in this manual. The laboratory template project should be accessible after executing the setup script as follows:

```
<hls-lab>: $HOME
└─ lab-module-3
   └─ sw (main_sw.c main_hw.c main_hw_dma.c)
      └─ vp
         └─ hls
            ├── aes_ha_basic_prj
            └─ aes_ha_dma_prj
```

For every new terminal, you should source the environment before interacting with any toolchains or executables ¹:

0. Source the environment:

```
source $HOME/lab-module-3/environment.sh
```

Build the VP

0. As usual and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Configure your VP-build in a **build** directory:

```
cmake -S $VP_DIR -B build
```

¹you can confirm that the environment is correctly sourced, by the command prompt displaying (`venv`).

3. Build the VP with Make (via CMake):

```
cmake --build build
```

Build the Target Software

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the target software directory:

```
cd $HOME/lab-module-3/sw
```

2. Configure your SW-build in a build directory:

```
cmake -S . -B build
```

3. Build the target software with Make (via CMake) (-j8 to parallelize the build):

```
cmake --build build (-j8)
```

The cross-compiled binaries/ELFs `sw.elf` and `hw.elf` can be found in the build directory. Each time you make changes to your application software, you have to rebuild your binary (step 3). If you add new sources, you need to adapt the `CMakeLists.txt` and invoke `cmake` (step 2) again. Usually, all necessary files are provided and all software development tasks can be handled inside the applications' source files `main_sw.c` and `main_hw.c` for the pure software (`sw.elf`) and hardware-accelerated system (`hw.elf`), respectively.

3.1 Laboratory Tasks

3.1.1 A) Application Profiling of Pure Software Implementation (AES-SW)

Before building dedicated hardware for the AES task, we will check whether there is even a need or potential for offloading the task to the CPU. In general, it depends on the resource utilization (CPU) and whether there is a need for additional computational resources or dedicated hardware. This is also called Design Space Exploration (DSE). For this laboratory, we assume that our SoC will be integrated into an embedded system and has to decrypt a fixed-size message within 15 ms. The message comes with the following specs:

- The encrypted message may be represented as a byte matrix with 46 rows of 192 columns. Each byte may be displayed as an ASCII character.
- a 128-bit public key (which will be used as the CTR's initialization vector (IV))

3 Lab Module 3: Advanced Encryption Standard

To simplify the task, the information was already received and pre-processed. This is simulated in the source code by having the message instantiated as an existing global variable `msg`, the public key as a 16-byte array and the private key already retrieved from the device by secure means ².

Check your Setup

There are different ways to use and debug the VP. To keep this manual manageable, debugging is not thoroughly covered and we look only for the printed outputs (VP terminal and UART terminal). Refer to this laboratory's HelloWorld tutorial (Lab1) for tips on more sophisticated debugging techniques.

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Start the VP via command line. The command line arguments `-i` specify configurations for ETISS and EDAdduino:

```
build/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \  
--elfs ../sw/build/sw.elf
```

3. Open a second terminal, change into VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp  
cat uartout
```

It may be necessary to execute the VP first, so that the Posix FIFO in `uartout` is generated.

You should now see some information about the simulation environment in the VP terminal, as well as the incomprehensible output in the UART terminal. The latter is the still-encrypted message, which we will now decrypt.

Complete the Source Code

The target software's source code lacks the decryption functionality. Launch your preferred text or source code editor (or use the IDE), and complete the missing parts in: `$HOME/lab-module-3/sw/main_sw.c`

As usual, the missing part is marked with a `TODO` comment along with some tips inside the `crypt_msg_sw` function body. Also, have a look at the provided source files at

²we do not cover cryptography itself within this course. Please note that our HW/SW prototype is not a "complete system"

3 Lab Module 3: Advanced Encryption Standard

sw/aes/aes.{c,h}, to understand what the AES software library expects as inputs. The overall goal is to correctly invoke the bidirectional AES-CTR crypto function and return the execution time. You can also take a look at the software libraries' open-source repository for further hints [1].

```
1 ///////////////////////////////////////////////////////////////////
2 /// \brief Bi-directional (Encrypt/Decrypt) using the software library (app/aes.h)
3 /// \param msg Message as 1-dimensional byte-array, accepts msg_t format, too.
4 /// \return amount of cycles spent for en-/decrypting the message
5 uint32_t crypt_msg_sw(uint8_t *msg, const uint8_t *key, const uint8_t *iv)
6 {
7     uint64_t cycle_cnt_start = 0, cycle_cnt_end = 0;
8     struct AES_ctx ctx;
9     /* TODO: Invoke the software en-/decrypt a msg_t data structure. Return the spent time
10      * for setup and actual
11      * en-/decryption Tips:
12      * - AES_CTR_xcrypt_buffer(..) takes a AES_ctx struct instantiated by AES_init_ctx_iv
13      *   (..) the key and the IV, a
14      * read-write byte-array buffer (message), and the length of the buffer to en-/decrypt.
15      * - The IV is updated inside the AES_ctx struct within one call to
16      * AES_CTR_xcrypt_buffer(..)
17      * - AES_CTR_xcrypt_buffer(..) takes messages with lengths of 16-byte multiplicity
18      * - Use get_cpu_cycles() from csp.h to retrieve the current CPU cycle count */
19     return (cycle_cnt_end-cycle_cnt_start);
20 }
```

Listing 3.1: AES-SW crypto function

Once you have finished your software development, rebuild the target software and test on the VP. You can avoid jumping between directories by invoking the target software's build process from within the VP directory. For this section, normally, you do not need to rebuild the VP executable:

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Build the target software.

```
cmake --build ../sw/build
```

3. Open a second terminal, change into the VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp
cat uartout
```

4. Start the VP via the command line. The command line arguments `-i` specify configurations for ETISS and EDAdino:

```
build/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \  
--elfs ../sw/build/sw.elf
```

(opt.) You can also run the UART read command `cat` in the background of the VP executable command, such that you do not have to open a new terminal:

```
cat uartout & build/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \  
--elfs ../sw/build/sw.elf
```

The UART terminal should now display the decrypted message and the total number of decryption cycles. Calculate the real-time from this cycle count ³.

Let us consider an execution time of 50 ms: This exceeds our constraint for 15 ms by a factor of 3 and is unacceptable. Although there is not so much variance in the original message, i.e., repetitive blocks, the BCMO encrypting the message ensured a higher variance in the encrypted message.

3.1.2 B) AES Hardware Accelerator without Direct Memory Access

High-Level Synthesis: AES-SW to AES-HA

Since we observed that EDAduino's processing system cannot complete the allocated task within the 15 ms constraint, we will attempt to determine whether we can accelerate encryption and decryption with dedicated hardware. For this, we utilize Vivado-HLS. Refer to the `HLSIntroTutorial` for basic Vivado-HLS usage and the `DesignAnalysisTutorial` for possible optimization techniques. As a goal, the following properties should hold for the resulting solution to later ensure correct interfacing with the VP:

1. Do not change or introduce additional directives to the top-level function's interface. Only *AMBA AXI4-Lite* interface is supported by our VP.
2. the design is implemented for the Xilinx FPGA chip `xc7s15-ftgb196-2`,
3. no single resource usage shall exceed 100%,
4. a maximum latency of 500 clock cycles per 128-bit block, and
5. the solution must pass the Vivado-HLS *C/RTL Cosimulation* test)

To simplify the task, the project folder contains a pre-configured Vivado-HLS project, including the fixed-interface top-level function. Optimization through HLS directives can be applied to the various functions in `aes.c`. Note that the AES software library provides more than the CTR implementation used here. Thus, some sections are not in the top function's (`aes_ha`) hierarchy. Fig. 3.5 shows the top function's call graph. Nodes represent a function of the AES software library, and edges represent the expected number of calls to a function.

³The CPU frequency is 100 MHz

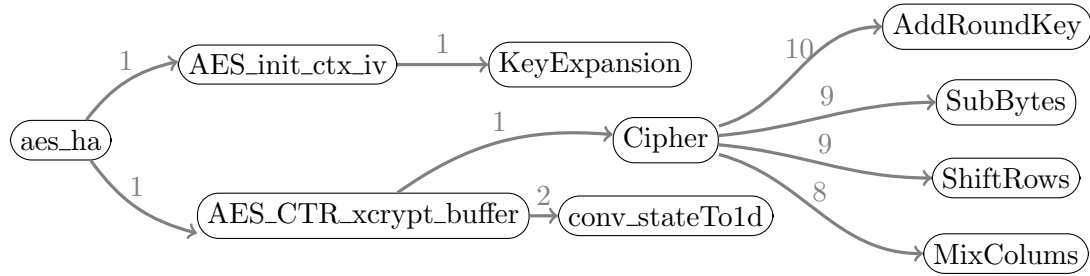


Fig. 3.5: CRT AES-HLS call graph (one block cipher)

Introduction

You should have a personal copy of the Vivado-HLS project (`aes_ha_basic_prj`) and some other files related to the VP integration flow in the HLS directory `lab-module-3/hls`.

0. Source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Launch Vivado-HLS and open the project directory

```
$HOME/lab-module-3/hls/aes_ha_basic_prj:
```

```
vivado_hls
```

2. **Run the C Simulation.** This compiles the software library and top function invoked by the provided testbench with GCC and tests the reported values against a "golden" reference.
3. **Synthesize Solution so11.** This default solution has no directives applied to the software library. The report indicates that the overall latency ranges from 446 to 22846 cycles. Vivado-HLS cannot determine the exact latency, since the `AES_CTR_xcrypt_buffer` function normally accepts input of varying sizes. However, in the Analysis-View, the latency of `Cipher` is shown: 1250 cycles. From this we can derive that the lower bound latency of `aes_ha` comes from an estimated input length of 0 bytes not 16 bytes as we have configured the call to `AES_CTR_xcrypt_buffer` to be⁴. This violates our design goal (4): maximum latency of 500 cycles. According to the Utilization Estimates, we have not exceeded any resources.
4. **Synthesize Solution so12.** In the Synthesis view, double-click the folder `so12` in the Explorer. This default solution has the aggressive `PIPELINE` directive applied to the top function. The generated Synthesis report informs us of an overall latency of 231 cycles. However, the Utilization Estimate for Look-Up Tables (LUTs) is exceeded by 15% (total of 115%). This violates our design goal (3): maximum resource utilization is 100%.

⁴Another way to determine the top level latency and interval is to analyze the trace generated from a C/RTL Co-simulation by measuring the number of clock cycles between *start* and *done* events.

Reach the Design Goals

The provided default solutions `sol1` and `sol2` both violate our design goals. Your task is to develop your solution that holds our constraints. There are no rules for the directives applied as long as they are not applied to the top function interface `aes_ha`.

1. **Create a new solution `sol3`.**

In Vivado-HLS, click *Menu:Project->New Solution*

Set the solution name to `sol3`, period to 10 (in nanoseconds), uncertainty to 12.5%, and select Option *Copy directive and constraints from solution: sol1*.

2. Make `sol3` the active solution (double click on the folder in Explorer).

3. In Synthesis View, open `aes.c` source file (Explorer->Source). Apply directives to components (functions, variables, loops, etc.) of functions depicted in the call graph (Fig. 3.5). Feel free to explore the various directives Vivado-HLS supports. For the most part, `DATAFLOW`, `PIPELINE`, `INLINE`, and `UNROLL`, i.e., the ones introduced in the previous lab, should suffice. However, you might come up with a different or even better solution than any reference design.

4. **Synthesize your solution.** If the generated report fits the design goals, you can proceed with step 5. Otherwise, try to optimize your directives further.

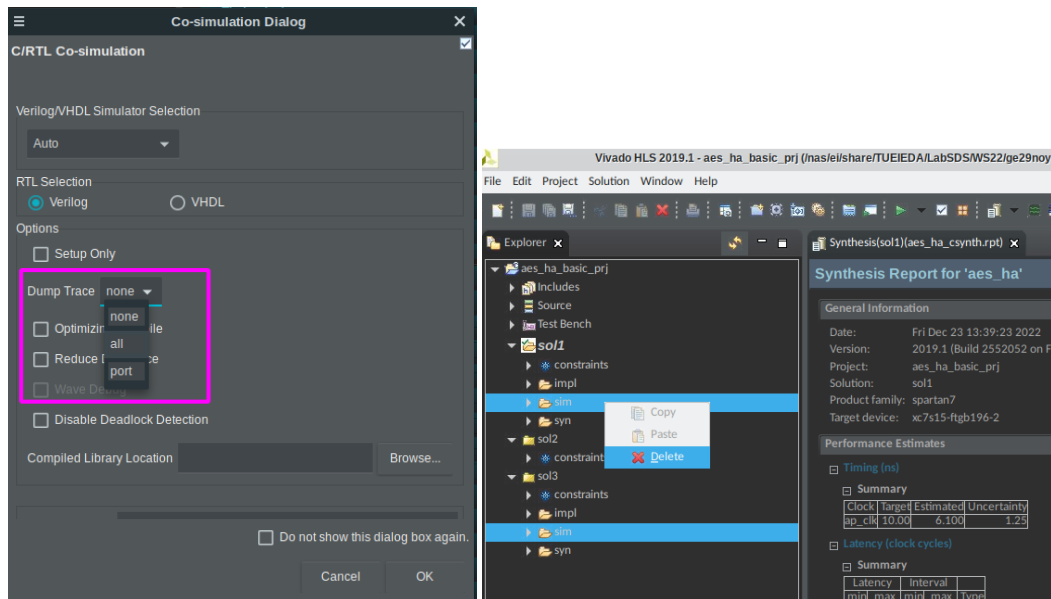
5. **Run C/RTL Cosimulation.** Click *Menu: Solution->Run C/RTL Co-simulation*

The Simulation should report a passed test at the end. Otherwise, although unlikely, your directives might have broken the algorithm's functionality. This can happen if you introduce directives altering the analyzed data dependency, e.g., the `DEPENDENCY` directive. If unsure about the latency, you can invoke the C/RTL Co-simulation with dump tracing set to *all*. After completing Co-simulation, open the wave viewer in Vivado:

Click *Menu: Solution->Open Wave Viewer*. The synthesized testbench encrypts one block and then decrypts it again such that you have to measure the distance between the start and done signals⁵. The testbench clock frequency is 100 MHz. For example, if you measure $45.1\mu s$, 4510 clock cycles have passed. Should you experience "Disk quota" error messages, try to delete the `sim/` directories for your solutions (simulation data takes up $\sim 500MB$).

⁵these signals are available in the *Block-level IO Handshake (internal)* group

3 Lab Module 3: Advanced Encryption Standard



Build the AES-HA VRTL

Once you are satisfied with your solution `sol3` and it meets the design goals, you usually export the RTL as an IP to the main Vivado tools. You would implement it in an FPGA design and generate a bitstream for a physical FPGA chip. Our implementation will be a Verilated RTL (VRTL) for the EDAduino VP's SystemC environment, as described in Sec. 3.0.3.

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the AES-HLS directory:

```
cd $HOME/lab-module-3/hls
```

2. **Invoke the GMake build process:** This launches the Verilation process on the specified solution, builds a custom SystemC testbench, and runs the SystemC test. The SystemC test's input set is equivalent to the tests performed at earlier stages but is also tailored to the interface and timing expected by the later TLM Transactor. Replace `<solution-name>` with the name of the solution you want to implement, e.g., `sol3` for your solution or `sol1` and `sol2` for the provided ones in case you want to work on the VP side before looking into the HLS part. The `sol1` might be a bit buggy. So don't be scared.

```
make SOL=<solution name>
```

Build the VP (AES-HA without DMA in EDAduino)

We must reconfigure the simulation to integrate the accelerator into our SystemC Virtual Prototype.

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Configure your VP-build in a build directory. AESVRTL_PREFIX specifies the location of your compiled executable of the previous task, i.e., the AES-HA VRTL:

```
cmake -S $VP_DIR -B build-nodma \  
-D AESVRTL_PREFIX=$HOME/lab-module-3/hls -D AES_TYPE=basic
```

3. Build the VP with Make (via CMake):

```
cmake --build build-nodma
```

The AES-HA solution used (`sol1`, `sol2`, `sol3`, ...) is always the latest build in the `hls/` directory. You can swap between solutions by invoking a different VRTL build (Sec. 3.1.2) and then rebuilding the VP (Sec. 3.1.2).

Build the Target Software

The hardware-accelerated software is compiled simultaneously with the pure software-based one. Make changes to `main_hw.c` and invoke the cross-compiler toolchain in the `sw/` directory:

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/sw
```

2. Build the target software.

```
cmake --build build
```

The cross-compiled binary/ELF `hw.elf` can be found in the build directory. Each time you change your application software, you must rebuild your binary (step 2). If you add a new source, you need to adapt the `CMakeLists.txt` and invoke `cmake` (step 1) again. Usually, all necessary files are provided, and all software development tasks can be handled inside the application's `main_hw.c`.

Check your Setup

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Open a second terminal, change into the VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp
cat uartout
```

You could have to first execute the VP once, so the Posix FIFO in `uartout` is generated.

3. Start the VP via the command line. The command line arguments `-i` specify configurations for ETISS and EDAduino:

```
build-nodma/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \
--elfs ../sw/build/hw.elf
```

Again, like in the software part, you should see some information about the simulation environment in the VP terminal and incomprehensible output in the UART terminal. The latter is the still-encrypted message. The former should also report the result of a quick test of the AES-HA functionality, running an en and decryption on a 16-byte block. A brief self-check of the hardware accelerator is performed before that.

Complete the Source Code

The target software's source code lacks code that invokes the hardware accelerator. Launch your preferred text or source code editor (or use the IDE), and complete the missing parts in `$HOME/lab-module-3/sw/main_hw.c`.

As usual, the missing part is marked with a `TODO` comment along with some tips inside the `crypt_msg_hw` function's body. Also, have a look at the provided driver files to understand what the AES-HA driver expects as inputs (see `sw/xil/basic/xaes_ha.{c,h}`). The goal is to correctly invoke the bidirectional AES-HA through the generated driver functions and return the time difference between setup and finished en-/decryption. You can also take a look at the `aes_ha_test` (in `sw/aes/aes_ha_drv.c`), performing a simple functional test on a single block to get an idea about the driver functions and their usage ⁶ **Optional:** The minimal functional test does not use interrupts. You can choose to do so.

⁶Please also note that AES-HA automatically increments the IV register's value for each processed block. You only have to write IV once for a block sequence.

3 Lab Module 3: Advanced Encryption Standard

```
1 volatile int aes_done = 0; ///< Global variable for simple messaging from AES Interrupt
   Service Routine to main
2                               ///< application that a "Done" event was issued from the
   hardware accelerator
3 volatile int aes_ready = 0; ///< Global variable for simple messaging from AES Interrupt
   Service Routine to main
4                               ///< application a "Ready" event was issued from the hardware
   accelerator
5 volatile int aes_iir = 0; ///< Latest Interrupt identification register state
6 //////////////////////////////////////
7 ///< \brief AES_HA ISR
8 static void aes_ha_irqhandler(void)
9 {
10     aes_iir = XAes_ha_InterruptGetStatus(&iaes);
11     if (aes_iir & 1)
12     {
13         XAes_ha_InterruptClear(&iaes, 1);
14         aes_done = 1;
15         // custom_print_string((void *)ETISS_LOGGER_ADDR, "\nAES done\n");
16     }
17     if (aes_iir & 2)
18     {
19         XAes_ha_InterruptClear(&iaes, 2);
20         aes_ready = 1;
21         // custom_print_string((void *)ETISS_LOGGER_ADDR, "\nAES ready\n");
22     }
23 }
24
25 //////////////////////////////////////
26 ///< \brief Bi-directional (Encrypt/Decrypt) using the hardware accelerator
27 ///< \param msg Message as 1-dimensional byte-array, accepts msg_t format, too.
28 ///< \return amount of cycles spent for en-/decrypting the message
29 uint32_t crypt_msg_hw(uint8_t *msg, const uint8_t *key, const uint8_t *iv)
30 {
31     uint64_t cycle_cnt_start = 0, cycle_cnt_end = 0;
32     /* TODO: Invoke the hardware accelerator to en-/decrypt a msg_t data structure. Return
   the spent time for setup and
33     * actual en-/decryption Tips: - Byte-Wise driver functions (see xil/basic/xaes_ha.h)
   take a pointer to a byte-array
34     * and accesses the first 128-bit or rather 16 Byte, or 1 AES-Block. Word-wise driver
   functions interpret the given
35     * pointer as 4-byte elements. See (aes/aes_ha_drv.{c}'s sanity check for a minimal
   usage example.)
36     * - After every en-/decryption, the IV-register in hardware is updated according to
   the CTR logic. The IV
37     * register inside the accelerator is updated after each block automatically.
38     * - The key register is not updated by the hardware.
39     * - Find a way to determine a finished 128-bit encryption by interfacing with the
   AES_HA ISR. The Interrupt
40     * configuration is already implemented.
41     * - Use get_cpu_cycles() from csp.h to retrieve the current CPU cycle count */
42
43     return (cycle_cnt_end - cycle_cnt_start);
44 }
```

Listing 3.2: AES-HA crypto function

Once you have finished your software development, rebuild the target software and test it on the VP. You can avoid jumping between directories by invoking the target software's build process from within the VP directory. For this section, normally, you do not need

3 Lab Module 3: Advanced Encryption Standard

to rebuild the VP executable:

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Build the target software.

```
cmake --build ../sw/build
```

3. Open a second terminal, change into the VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp
cat uartout
```

4. Start the VP via the command line. The command line arguments `-vp`, `-etiss` specify configurations for ETISS and EDAdduino:

```
build-nodma/edarduino --etiss ini/etiss.ini --vp ini/vp.ini \
--elfs ../sw/build/hw.elf
```

(opt.) You can also run the UART read command `cat` in the background of the VP executable command, such that you do not have to open a new terminal:

```
cat uartout & build-nodma/edarduino --etiss ini/etiss.ini --vp ini/vp.ini \
--elfs ../sw/build/hw.elf
```

The UART terminal should now display the decrypted message and the total number of decryption cycles. Calculate the real-time from this cycle count. Would the SoC now be able to work on the decrypted message within 15 ms of receiving it?

3.1.3 C) AES Hardware Accelerator with Direct Memory Access

AES-HA Integration

We synthesize an additional bus initiator socket to enable the hardware accelerator to manage its I/O operations independently of the CPU. In our case *AMBA AXI4-M* [3]. We have fixed the HLS top-level interface to synthesize a *AMBA AXI4-M* [4] for configuration and a *AMBA AXI4-M* to enable direct memory access (DMA). The transactor consumes incoming memory transactions from its TLM target socket, e.g., from the CPU, and generates the required *AMBA AXI4-Lite* signals for the HA's input ports. The *AMBA AXI4-M* signals are scanned and translated into TLM transactions placed on the bus through the initiator socket. The *AMBA AXI4-Lite* target (the HA) can be programmed to trigger an interrupt on the standard events "Ready" and "Done".

”Done” means that one processed message has been entirely placed in memory. ”Ready” means a new message may be processed. The CTR IV is stored in internal registers and is both readable and writable by the CPU for initialization and incremented by the AES-HA after each 128-bit block is finished. Fig. 3.6 depicts a block diagram of the HA integration as seen from the EDAdiuno bus.

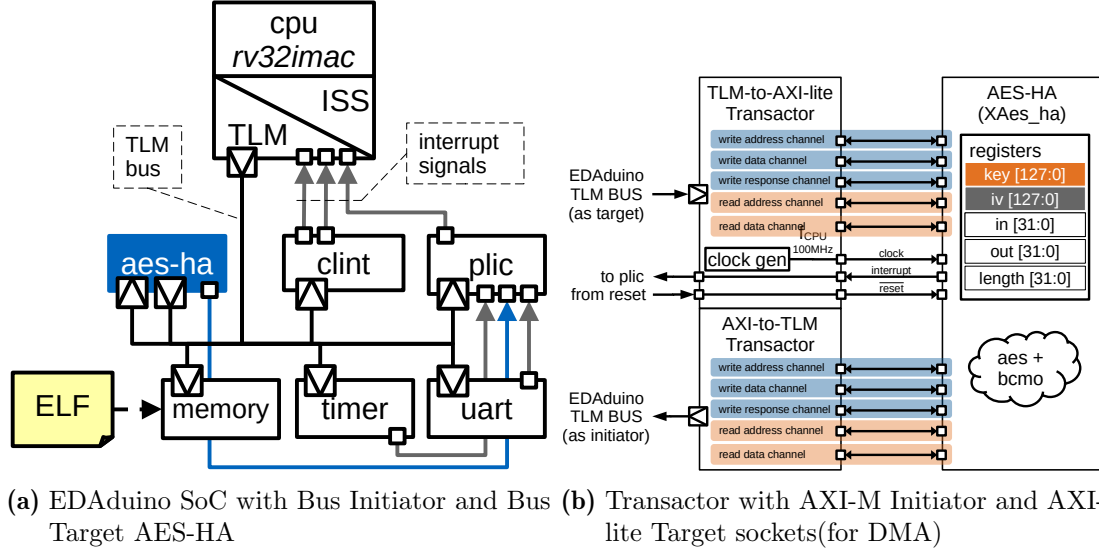


Fig. 3.6: Block model of DMA AES-HA and TLM-Transactor in EDAdiuno VP

The low-level software drivers for the standardized *AMBA AXI4-Lite* interface are similar to what was introduced in Section 3.0.3. The main difference is that now the I/O is managed by the hardware accelerator. The CPU can configure the start memory address of the input and output via respective registers. The length of the message in bytes is also configured via register.

High-Level Synthesis: AES-SW to AES-HA

Since we observed that EDAdiuno’s processing system cannot complete the allocated task within the 15 ms constraint, we will attempt to determine whether we can accelerate encryption and decryption with dedicated hardware. For this, we utilize Vivado-HLS. Refer to the HLSIntroTutorial for basic Vivado-HLS usage and the DesignAnalysisTutorial for possible optimization techniques. As a goal, the following properties should hold for the resulting solution to later ensure correct interfacing with the VP:

1. Do not change nor introduce additional directives to the top-level function’s interface. Only *AMBA AXI4-Lite* interface and *AMBA AXI4-M* are supported by our VP.
2. the design is implemented for the Xilinx FPGA chip xc7s15-ftgb196-2,
3. no single resource usage shall exceed 100%,

3 Lab Module 3: Advanced Encryption Standard

4. a maximum latency of 500 clock cycles per 128-bit block, and
5. the solution must pass the Vivado-HLS *C/RTL Cosimulation* test)

To simplify the task, the project folder contains a pre-configured Vivado-HLS project, including the fixed-interface top-level function. Optimization through HLS directives can be applied to the various functions in `aes.c`. Note that the AES software library includes more than the used CTR implementation; therefore, some sections are not within the top function's (`aes_ha`) hierarchy.

You should have a personal copy of the Vivado-HLS project (`aes_ha_dma_prj`) and some other files related to the VP integration flow in the HLS directory `lab-module-3/hls`.

0. Source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Launch Vivado-HLS and open the project directory

```
$HOME/lab-module-3/hls/aes_ha_dma_prj:
```

```
vivado_hls
```

2. **Run the C Simulation.** This compiles the software library and top function invoked by the provided testbench with GCC and tests the reported values against a "golden" reference.
3. **Synthesize Solution sol1.** This default solution has no directives applied to the software library. Vivado-HLS Synthesis cannot determine the top-level latency because the message length is configurable. However, in the Analysis-View, the latency of `Cipher` is shown: 1250 cycles. From this, we can derive that the lower bound latency of `aes_ha` comes from an estimated input length of 0 bytes, not 16 bytes, as we have configured the call to `AES_CTR_xcrypt_buffer` to be⁷. This violates our design goal (4): maximum latency of 500 cycles for each block. According to the Utilization Estimates, we have not exceeded any resources.
4. **Synthesize Solution sol2.** In the Synthesis view, double-click the folder `sol2` in the Explorer. This default solution has the aggressive `PIPELINE` directive applied to the top function. Again, the generated Synthesis report can not specify the top-level latency. In the Analysis, we can see that the `KeyExpansion` and `Cipher` meet the design goals. However, the solution exceeds maximum resource utilization (LUTs at 160%).

Reach the Design Goals

The provided default solutions `sol1` and `sol2` violate our design goals. Your task is to develop a solution that meets our constraints and aligns with our design goals. There

⁷Another way to determine the top-level latency and interval is to analyze the trace generated from a C/RTL Co-simulation. However, the "Done" and "Ready" now indicate a message being fully processed, not a single 128-bit block, which allows us to calculate an average time per block. If you use the Simulation Report, be aware that the testbench processes the message twice.

3 Lab Module 3: Advanced Encryption Standard

are no rules for the directives applied as long as they are not used to the top function interface `aes_ha`. You can try to reuse directives found in the previous target-only hardware accelerator.

Build the AES-HA VRTL

Once you are satisfied with your solution `sol3` and it meets the design goals, you usually export the RTL as an IP to the main Vivado tools. You would implement it in an FPGA design and generate a bitstream for a physical FPGA chip. Our implementation will be a Verilated RTL (VRTL) for the EDAdino VP's SystemC environment, as described in Section 3.0.3.

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the AES-HLS directory:

```
cd $HOME/lab-module-3/hls
```

2. **Invoke the GMake build process:** Replace `<solution-name>` with the name of the solution you want to implement, e.g., `sol3` for your personal solution or `sol1` and `sol2` for the provided ones in case you want to work on the VP side before looking into the HLS part. Make sure to add the argument `TYPE=dma` this time.

```
make SOL=<solution name> TYPE=dma
```

Build the VP (AES-HA with DMA in EDAdino)

We must reconfigure the simulator to integrate the DMA-based accelerator into our SystemC Virtual Prototype.

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Configure your VP-build in a `build` directory. `AESVRTL_PREFIX` specifies the location of your compiled executable of the previous task, i.e., the AES-HA VRTL. `AES_TYPE=dma` hints the VP to configure the initiator socket:

```
cmake -S $VP_DIR -B build-dma \  
-D AESVRTL_PREFIX=$HOME/lab-module-3/hls -D AES_TYPE=dma
```

3 Lab Module 3: Advanced Encryption Standard

3. Build the VP with Make (via CMake):

```
cmake --build build-dma
```

The AES-HA solution used (`sol1`, `sol2`, `sol3`, ...) is always the latest build in the `hls/` directory. You can swap between solutions by invoking a different VRTL build (Section 3.1.3) and then rebuilding the VP (Section 3.1.3).

Build the Target Software

The hardware-accelerated software is built simultaneously with the pure software-based one. Make changes to `main_hw.c` and invoke the cross-compiler toolchain in the `sw/` directory:

0. As usual and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/sw
```

2. Build the target software.

```
cmake --build build
```

The cross-compiled binary/ELF `hw_dma.elf` can be found in the build directory. Each time you make changes to your application software, you have to rebuild your binary (step 2). If you add a new source, you need to adapt the `CMakeLists.txt` and invoke `cmake` (step 1) again. Usually, all necessary files are provided, and all software development tasks can be handled inside the application's `main_hw_dma.c`.

Check your Setup

0. As usual and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Open a second terminal, change into VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp
cat uartout
```

3 Lab Module 3: Advanced Encryption Standard

It may be necessary to execute the VP first, so that the Posix FIFO in `uartout` is generated.

3. Start the VP via the command line. The command line arguments `-i` specify configurations for ETISS and EDAduino:

```
build-dma/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \  
--elfs ../sw/build/hw_dma.elf
```

- (opt.) You can also run the UART read command `cat` in the background of the VP executable command, such that you do not have to open a new terminal:

```
cat uartout & build-dma/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \  
--elfs ../sw/build/hw_dma.elf
```

Again, like in the software part, you should see some information about the simulation environment in the VP terminal and incomprehensible output in the UART terminal. The latter is the still encrypted message, and the former should also report the result of a quick test of the AES-HA functionality, running an encryption and decryption on a 16-byte block. Moreover, a brief self-check of the hardware accelerator is performed beforehand.

Complete the Source Code

The target software's source code lacks code that invokes the hardware accelerator. Launch your preferred text or source code editor (or use the IDE), and complete the missing parts in `$HOME/lab-module-3/sw/main_hw_dma.c`.

As usual, the missing part is marked with a `TODO` comment along with some tips inside the `crypt_msg_hw` function's body. Also, have a look at the provided driver files to understand what the AES-HA driver expects as inputs (see `sw/xil/dma/xaes_ha.{c,h}`). The goal is to correctly invoke the bidirectional AES-HA through the generated driver functions and return the time difference between setup and finished en-/decryption. You can also take a look at the `aes_ha_test` (in `sw/aes/aes_ha_drv.c`), performing a simple functional test on a single block to get an idea about the driver functions and their usage⁸. **Optional:** The minimal functional test does not make use of interrupts. You can choose to do so.

```
1 ///////////////////////////////////////////////////////////////////  
2 /// \brief Bi-directional (Encrypt/Decrypt) using the hardware accelerator  
3 /// \param msg Message as 1-dimensional byte-array, accepts msg_t format, too.  
4 /// \return the amount of cycles spent for en-/decrypting the message  
5 uint32_t crypt_msg_hw(uint8_t *msg, const uint8_t *key, const uint8_t *iv)  
6 {  
7     uint64_t cycle_cnt_start = 0, cycle_cnt_end = 0;  
8     /* TODO: Invoke the hardware accelerator to en-/decrypt a msg_t data structure. Return  
       the spent time for setup and
```

⁸Please also note that AES-HA automatically increments the IV register's value for each processed block. This means you only have to write IV once for a block sequence

3 Lab Module 3: Advanced Encryption Standard

```
9      * actual en-/decryption Tips: - Byte-Wise driver functions (see xil/dma/xaes_ha.h) take
      * a pointer to a byte-array
10     * and set, e.g., the 128-bit IV/Key. Word-wise, driver functions interpret the given.
11     * pointer as 4-byte elements. See (aes/aes_ha_drv.{c})'s sanity check for a minimal
      * usage example.)
12     * - After every en-/decryption, the IV-register in hardware is updated according to
      * the CTR logic. The IV
13     * register inside the accelerator is updated automatically after each block.
14     * - The hardware does not update the key register.
15     * - Find a way to determine a finished 128-bit encryption by interfacing with the
      * AES_HA ISR. The Interrupt
16     * configuration is already implemented.
17     * - Use get_cpu_cycles() from csp.h to retrieve the current CPU cycle count */
18     return (cycle_cnt_end - cycle_cnt_start);
19 }
```

Listing 3.3: AES-HA crypto function with DMA-enabled HA

Once you have finished your software development, rebuild the target software and test it on the VP. You can avoid jumping between directories by invoking the target software's build process from within the VP directory. For this section, normally, you do not need to rebuild the VP executable:

0. As usual, and if not already done, source the environment:

```
source $HOME/lab-module-3/environment.sh
```

1. Change into the VP directory:

```
cd $HOME/lab-module-3/vp
```

2. Build the target software.

```
cmake --build ../sw/build
```

3. Open a second terminal, change into VP directory, and listen to the UART:

```
cd $HOME/lab-module-3/vp
cat uartout
```

4. Start the VP via the command line. The command line arguments `-vp`, `-etiss` specify configurations for ETISS and EDAduino:

```
build-dma/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \
--elfs ../sw/build/hw_dma.elf
```

(opt.) You can also run the UART read command `cat` in the background of the VP executable command, such that you do not have to open a new terminal:

```
cat uartout & build-dma/edaduino --etiss ini/etiss.ini --vp ini/vp.ini \
--elfs ../sw/build/hw_dma.elf
```

The UART terminal should now display the decrypted message and the total number of decryption cycles. Calculate the real-time from this cycle count. Would the SoC now be able to work on the decrypted message within 15 ms of receiving it?

3.2 Questions (for exercise only)

1. Briefly mention the significance of application profiling in the context of SoC design.
2. What is High-Level Synthesis (HLS) and Model-Based Design (MBD)? Please list the advantages of each of them.
3. What is the key difference between application development with and without an OS?
4. What are the advantages of using the Hardware Accelerators (HAs) in an SoC?
5. How can HA especially be helpful for cryptography applications?
6. What are the limitations of high-level languages and C/C++ specifically that prevent hardware synthesizability?
7. Briefly highlight the individual roles of *Verilator*, *Vivado-HLS* and VPs in the context of system prototyping.
8. Briefly describe the HW/SW co-design methodology.
9. Briefly mention the role of hardware acceleration within the context of modern embedded-system design.
10. Briefly mention some benefits of using established software libraries as sources for HAs.
11. How was the information flow between AES-HA and the CPU synchronized?
12. What other synchronization techniques could be used to control the information and data flow between HAs and a processing unit?
13. Is it good practice to wait for the interrupt in a while-loop and waste the CPU resource? What could we do or introduce to the application software instead?
14. Briefly describe how you profile the Decryption task.
15. How much speedup was obtained using HW accelerated decryption over pure-SW implementation?
16. Three different AES-HA solutions (sol1, sol2, sol3) were tested with the same target software and drivers. Only the interface is fixed to *AMBA AXI4-Lite*. The Vivado-HLS synthesis reports latencies of 1802, 231, and 451 clock cycles for sol1, sol2, and sol3, respectively. However, a complete message decryption is timed at 6044 μs , 3263 μs , and 3661 μs . What could be the reason(s) for the solutions not scaling?

3.3 Appendix

3.3.1 FAQ

- **Q: Regarding resource constraints for AES-HLS.**
A: You must not exceed 100% utilization for any of the assumed hardware resources, e.g., BRAM, LUs, or FFs.

3 Lab Module 3: Advanced Encryption Standard

- **Q: In theory, a resource usage that exceeds 100% cannot be synthesized into an actual FPGA, right?**

A: Vivado-HLS will let you synthesize for more resources than available, but will report so. For FPGAs, the next step would be Implementation, where you would not be able to implement the excessive synthesis solution.

Bibliography

- [1] Tiny AES in C. <https://github.com/kokke/tiny-AES-c>. Accessed: 2020-06-18.
- [2] Accellera. SystemC. open-source. <https://github.com/accellera-official/systemc>. Accessed: 2022-12-05.
- [3] ARM. AMBA AXI and ACE Protocol Specification Version E AMBA AXI3 and AXI4 Protocol Specification. <https://developer.arm.com/documentation/ih10022/e/AMBA-AXI3-and-AXI4-Protocol-Specification?lang=en>. Accessed: 2022-12-05.
- [4] ARM. AMBA AXI and ACE Protocol Specification Version E Chapter 11. AMBA AXI4-Lite. <https://developer.arm.com/documentation/ih10022/e/AMBA-AXI4-Lite-Interface-Specification/AMBA-AXI4-Lite?lang=en>. Accessed: 2022-12-05.
- [5] RISC-V Foundation. The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2019121. Editors Andrew Waterman and Krste Asanovic. 12 2019. Accessed: 2020-04-27.
- [6] Infineon. AURIX Hardware Security Module. <https://www.infineon.com/cms/de/product/microcontroller/32-bit-tricore-microcontroller/aurix-security-solutions/>. Accessed: 2020-06-18.
- [7] RISC-V International. The RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Document Version 2021120. Editors Andrew Waterman, Krste Asanovic, and John Hauser. 12 2021. Accessed: 2020-04-27.
- [8] Kitware. The CMake Project. open source. <https://cmake.org/>. Accessed: 2020-04-27.
- [9] D. Mueller-Gritschneider, M. Dittrich, M. Greim, K. Devarajegowda, W. Ecker, and U. Schlichtmann. The Extendable Translating Instruction Set Simulator (ETISS) Interlinked with an MDA Framework for Fast RISC Prototyping. In *RSP '17: Proceedings of the 28th International Symposium on Rapid System Prototyping: Shortening the Path from Specification to Prototype*, 28th International Symposium on Rapid System Prototyping: Shortening the Path from Specification to Prototype, pages 79–84. Association for Computing Machinery, 2017.
- [10] TUM EDA. ETISS open-source. <https://github.com/tum-ei-eda/etiss>. Accessed: 2020-04-27.
- [11] TUM EDA. ETISS-SC open-source. <https://github.com/tum-ei-eda/etiss-sc>. Accessed: 2022-12-04.
- [12] VP-Vibes. VPV-Peripherals open-source. <https://github.com/VP-Vibes/VPV-Peripherals>. Accessed: 2022-12-05.

Bibliography

- [13] VP-Vibes, Minres. SystemC-Components (SCC). open-source. <https://github.com/VP-Vibes/SystemC-Components>. Accessed: 2022-12-05.
- [14] Wilson Snyder. Verilator. <https://www.veripool.org/wiki/verilator>. Accessed: 2020-06-18.
- [15] Xilinx. Vivado design suite tutorial: High-level synthesis. https://www.xilinx.com/support/documentation/sw_manuals/xilinx2019_1/ug871-vivado-high-level-synthesis-tutorial.pdf, 1 2019. UG871-2019-1.